

Лабораторная работа №5

Построение графиков

Дурдалыев Максат

Содержание

1	Введение	7
1.1	Цели и задачи	7
2	Выполнение лабораторной работы	8
2.1	Основные пакеты для работы с графиками в Julia	8
2.2	Опции при построении графика	14
2.3	Точечный график	21
2.4	Аппроксимация данных	27
2.5	Две оси ординат	31
2.6	Полярные координаты	33
2.7	Параметрический график	35
2.8	График поверхности	39
2.9	Линии уровня	47
2.10	Векторные поля	53
2.11	Анимация	56
2.12	Гипоциклоида	60
2.13	Errorbars	69
2.14	Использование пакета Distributions	82
2.15	Подграфики	86
2.16	Самостоятельное выполнение	99
3	Выводы	132
	Список литературы	133

Список иллюстраций

2.1	Задание функции	9
2.2	Задание функции	10
2.3	График функции, построенный при помощи <code>gr()</code>	11
2.4	График функции, построенный при помощи <code>pyplot()</code>	12
2.5	График функции, построенный при помощи <code>unicodeplots()</code>	13
2.6	График функции $\sin(x)$	15
2.7	График функции разложения исходной функции в ряд Тейлора	16
2.8	Графики исходной функции и её разложения в ряд Тейлора	17
2.9	Добавления опций для графика	19
2.10	Вид графиков после добавления опций при их построении	20
2.11	График десяти случайных значений на плоскости (простой точечный график)	22
2.12	График пятидесяти случайных значений на плоскости с различными опциями отображения (точечный график с кодированием значения размером точки)	24
2.13	График пятидесяти случайных значений в пространстве с различными опциями отображения (3-мерный точечный график с кодированием значения размером точки)	26
2.14	Пример функции	28
2.15	Пример аппроксимации исходной функции полиномом 5-й степени	30
2.16	Пример отдельно построенной траектории	32
2.17	График функции, заданной в полярных координатах	34
2.18	Параметрический график кривой на плоскости	36
2.19	Параметрический график кривой в пространстве	38
2.20	График поверхности (использована функция <code>surface()</code>)	40
2.21	График поверхности (использована функция <code>plot()</code>)	42
2.22	Сглаженный график поверхности	44
2.23	График поверхности с изменённым углом зрения	46
2.24	График поверхности, заданной функцией $g(x, y) = (3x + y^2) \sin(x) + \cos(y) $	48
2.25	Линии уровня	50
2.26	Линии уровня с заполнением	52
2.27	График функции $h(x, y) = x^3 - 3x + y^2$	54
2.28	Линии уровня для функции $h(x, y) = x^3 - 3x + y^2$	55
2.29	Статичный график поверхности	57
2.30	Анимированный график поверхности	59

2.31	Построение большой окружности гипоциклоиды	61
2.32	Большая окружность гипоциклоиды	62
2.33	Половина пути гипоциклоиды	64
2.34	Малая окружность гипоциклоиды	66
2.35	Малая окружность гипоциклоиды с добавлением радиуса	68
2.36	Зададим исходные значения	70
2.37	График исходных значений	71
2.38	График исходных значений с отклонениями	73
2.39	Поворот графика	75
2.40	Заполнение цветом	77
2.41	График ошибок по двум осям	79
2.42	График асимметричных ошибок по двум осям	81
2.43	Гистограмма, построенная по массиву случайных чисел	83
2.44	Гистограмма нормального распределения	85
2.45	Серия из 4-х графиков в ряд	87
2.46	Серия из 4-х графиков в сетке	89
2.47	Серия из 4-х графиков разной высоты в ряд	91
2.48	Объединение нескольких графиков в одной сетке	93
2.49	Объединение нескольких графиков в одной сетке	94
2.50	Разнообразные варианты представления данных	96
2.51	Демонстрация применения сложного макета для построения графиков	98
2.52	Решение задания №1	100
2.53	Решение задания №2	102
2.54	Решение задания №3	104
2.55	Решение задания №4	106
2.56	Решение задания №5	108
2.57	Решение задания №5	109
2.58	Решение задания №6	111
2.59	Решение задания №7	113
2.60	Решение задания №8	115
2.61	Решение задания №9	117
2.62	Решение задания №10	119
2.63	Решение задания №10	120
2.64	Решение задания №10	121
2.65	Решение задания №10	122
2.66	Решение задания №10	123
2.67	Решение задания №10	124
2.68	Решение задания №11	126
2.69	Решение задания №11	127
2.70	Решение задания №11	128
2.71	Решение задания №11	129
2.72	Решение задания №11	130

2.73	Решение задания №11	131
------	-------------------------------	-----

Список таблиц

1 Введение

1.1 Цели и задачи

Цель работы

Основная цель работы — освоить синтаксис языка Julia для построения графиков[2].

Задание

1. Используя Jupyter Lab, повторите примеры. При этом дополните графики обозначениями осей координат, легендой с названиями траекторий, названиями графиков и т.п.
2. Выполните задания для самостоятельной работы[1].

2 Выполнение лабораторной работы

2.1 Основные пакеты для работы с графиками в Julia

Julia поддерживает несколько пакетов для работы с графиками. Использование того или иного пакета зависит от целей, преследуемых пользователем при построении. Стандартным для Julia является пакет Plots.jl.

Рассмотрим построение графика функции $f(x) = (3x^2 + 6x - 9)e^{-0,3x}$ разными способами (рис. [fig:001] - рис. [fig:005]):

```
# подключаем для использования Plots:
using Plots

[4] ✓ 382s Julia

...

SYSTEM: caught exception of type :MethodError while trying to print a failed Task notice; giving up

# задание функции:
f(x) = (3x.^2 + 6x .- 9).*exp.(-0.3x)

[14] Julia

... f (generic function with 1 method)

# генерирование массива значений x в диапазоне от -5 до 10 с шагом 0,1
# (шаг задан через указание длины массива):
x = collect(range(-5,10,length=151))

[15] Julia

... 151-element Vector{Float64}:
-5.0
-4.9
-4.8
-4.7
-4.6
-4.5
-4.4
-4.3
-4.2
-4.1
:
9.2
9.3
9.4
9.5
9.6
9.7
9.8
9.9
10.0
```

Рисунок 2.1: Задание функции

```

# генерирование массива значений y:
y = f(x)

[16] Julia

... 151-element Vector{Float64}:
 161.34080653217032
 146.26477779394003
 132.19219298833204
 119.06942359634911
 106.0453557470580
 95.47128180475011
 84.9007968362764
 75.00969810741056
 65.99589024347995
 57.579293095755176
  ⋮
 18.995125520005375
 18.811475185747092
 18.625664977689375
 18.437877278854756
 18.248288702120195
 18.057070179740368
 17.86438705526336
 17.6703991776229
 17.475260997120245

# указывается, что для построения графика используется gr():
gr()

[17] Julia

... Plots.GRBackend()
```

Рисунок 2.2: Задание функции

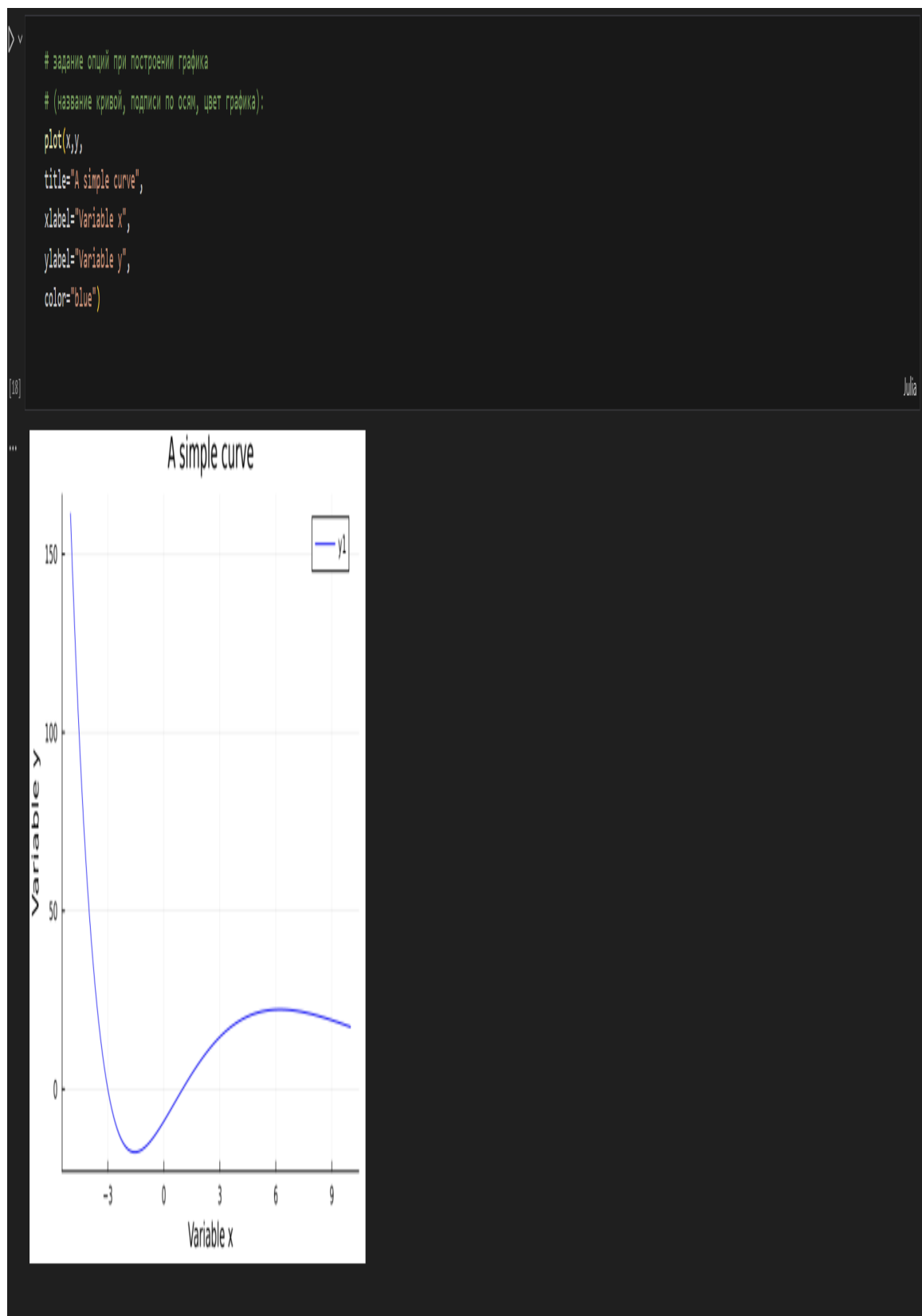


Рисунок 2.3: График функции, построенный при помощи `gr()`

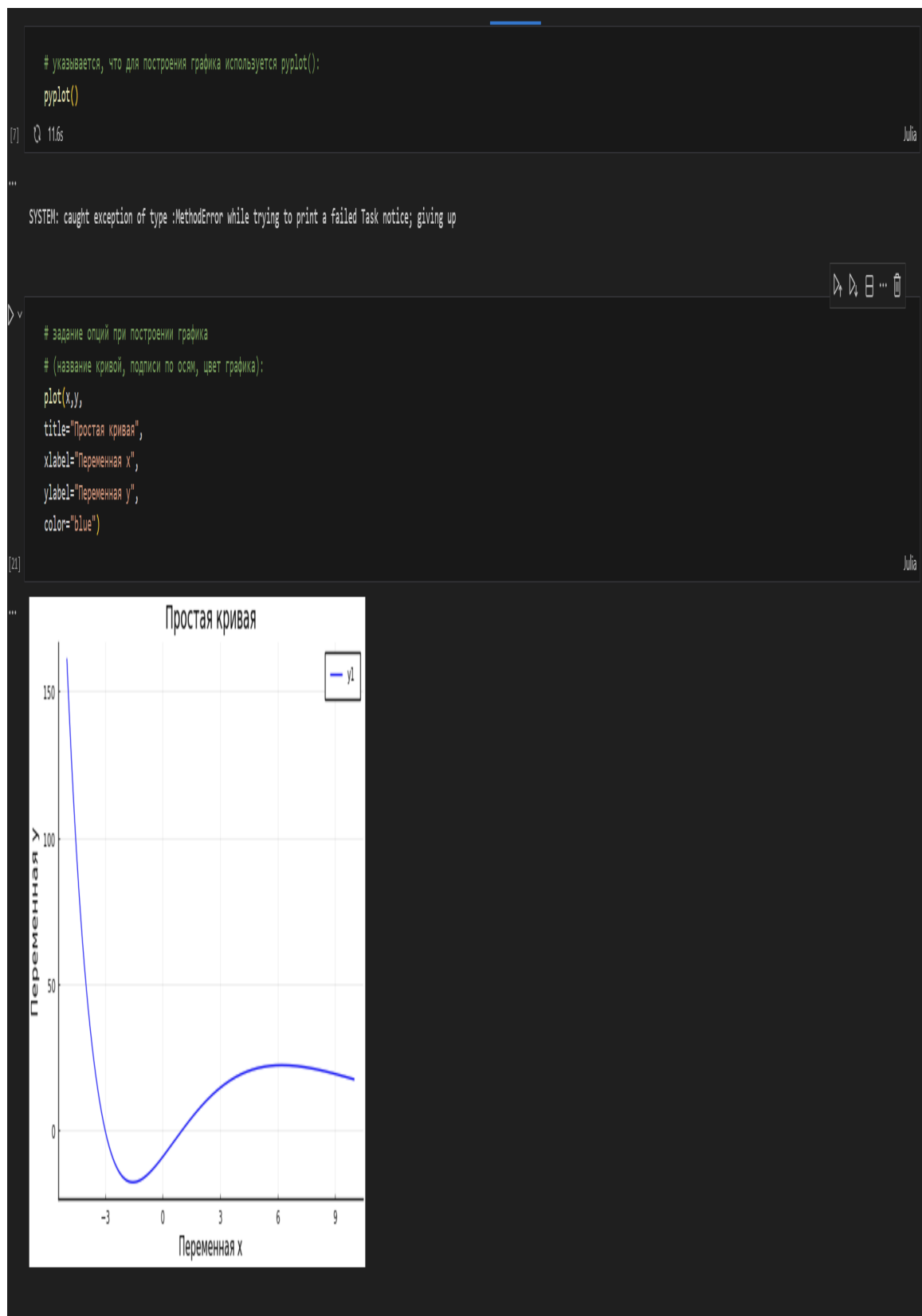


Рисунок 2.4: График функции, построенный при помощи pyplot()



Рисунок 2.5: График функции, построенный при помощи unicodeplots()

2.2 Опции при построении графика

На примере графика функции $\sin(x)$ и графика разложения этой функции в ряд Тейлора рассмотрим дополнительные возможности пакетов для работы с графикой (рис. [fig:006] - рис. [fig:008]):



Рисунок 2.6: График функции $\sin(x)$

```
# задание функции разложения исходной функции в ряд Тейлора:
sin_taylor(x) = [(-1)^i * x^(2*i+1) / factorial(2*i+1) for i in 0:4] |> sum
```

[44]

Julia

```
... sin_taylor (generic function with 1 method)
```

```
plot(sin_taylor,
      title="Ряд Тейлора для sin(x)",
      xlabel="Переменная x",
      ylabel="Переменная y",
      color="green",
      label = "sin_taylor")
```

[54]

Julia

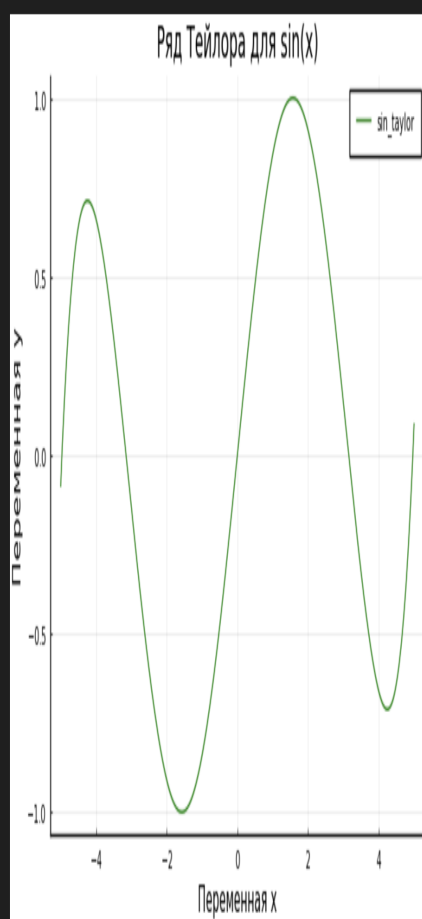


Рисунок 2.7: График функции разложения исходной функции в ряд Тейлора

```
# построение двух функций на одном графике:
```

```
plot(sin_theor)
```

```
plot!(sin_taylor)
```

[46]

Julia

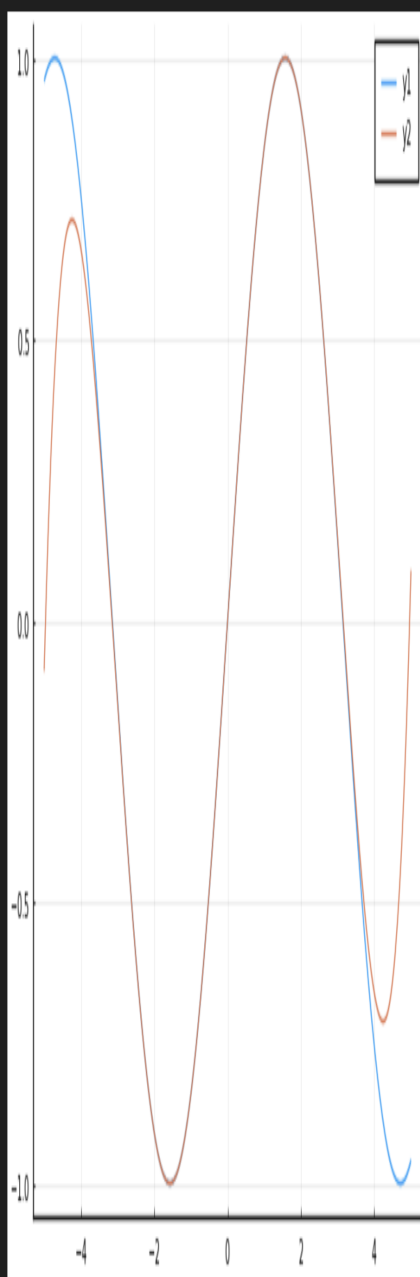


Рисунок 2.8: Графики исходной функции и её разложения в ряд Тейлора

Затем добавим различные опции для отображения на графике (рис. [fig:009] - рис. [fig:010]):

```
plot(
# функция sin(x):
sin_taylor,

# подпись в легенде, цвет и тип линии:
label = "sin(x), разложение в ряд Тейлора",
line=(blue, 0.3, 6, :solid),

# размер графика:
size=(800, 500),

# параметры отображения значений по осям
xticks = (-5:0.5:5),
yticks = (-1:0.1:1),
xtickfont = font(12, "Times New Roman"),
ytickfont = font(12, "Times New Roman"),

# подписи по осям:
ylabel = "y",
xlabel = "x",

# название графика:
title = "Разложение в ряд Тейлора",

# поворот значений, заданный по оси x:
xrotation = rad2deg(pi/4),

# заливка области графика цветом:
fillrange = 0,
fillalpha = 0.5,
fillcolor = :lightgoldenrod,

# задание цвета фона:
background_color = :ivory
)

plot(
# функция sin_theor:
sin_theor,

# подпись в легенде, цвет и тип линии:
label = "sin(x), теоретическое значение",
line=(black, 1.0, 2, :dash))
```

[47]

Julia

Рисунок 2.9: Добавления опций для графика

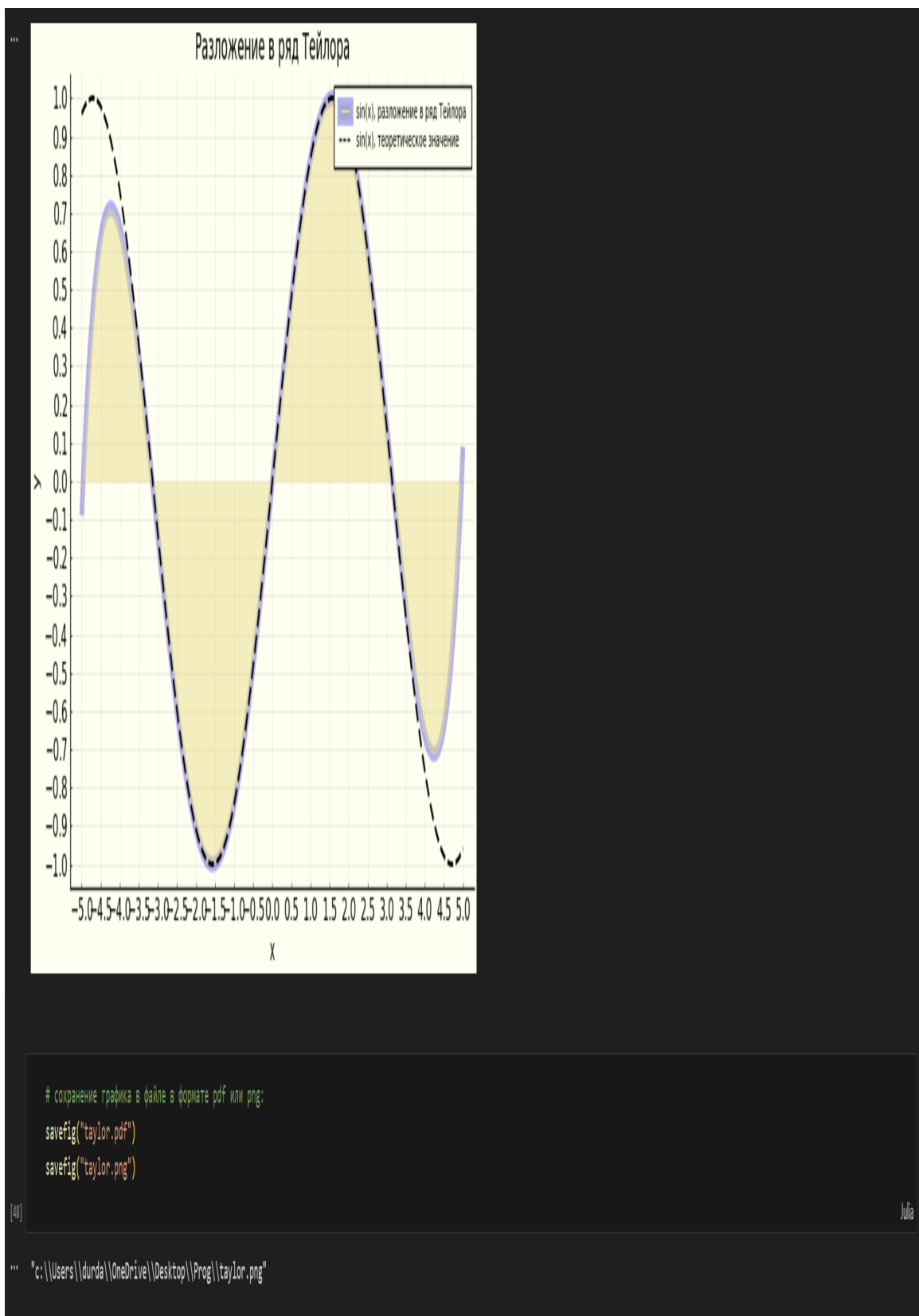


Рисунок 2.10: Вид графиков после добавления опций при их построении

2.3 Точечный график

Как и при построении обычного графика для точечного графика необходимо задать массив значений x , посчитать или задать значения y , задать опции построения графика (рис. [fig:011]):

```
# параметры распределения точек на плоскости:
```

```
x = range(1,10,length=10)
```

```
y = rand(10)
```

```
# параметры построения графика:
```

```
plot(x, y,
```

```
seriesType = :scatter,
```

```
title = "Точечный график"
```

```
)
```

[50]

Julia

...

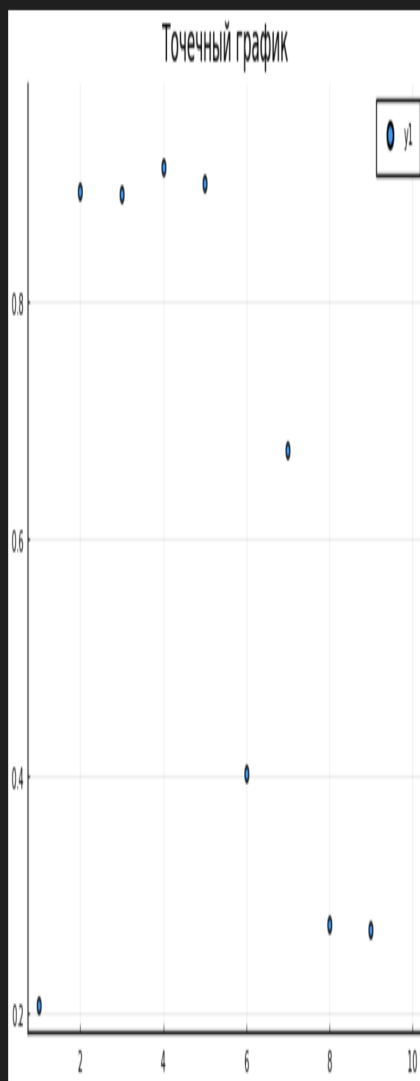


Рисунок 2.11: График десяти случайных значений на плоскости (простой точечный график)

Для точечного графика можно задать различные опции, например размер маркера, его тип, цвет и и т.п. (рис. [fig:012]):

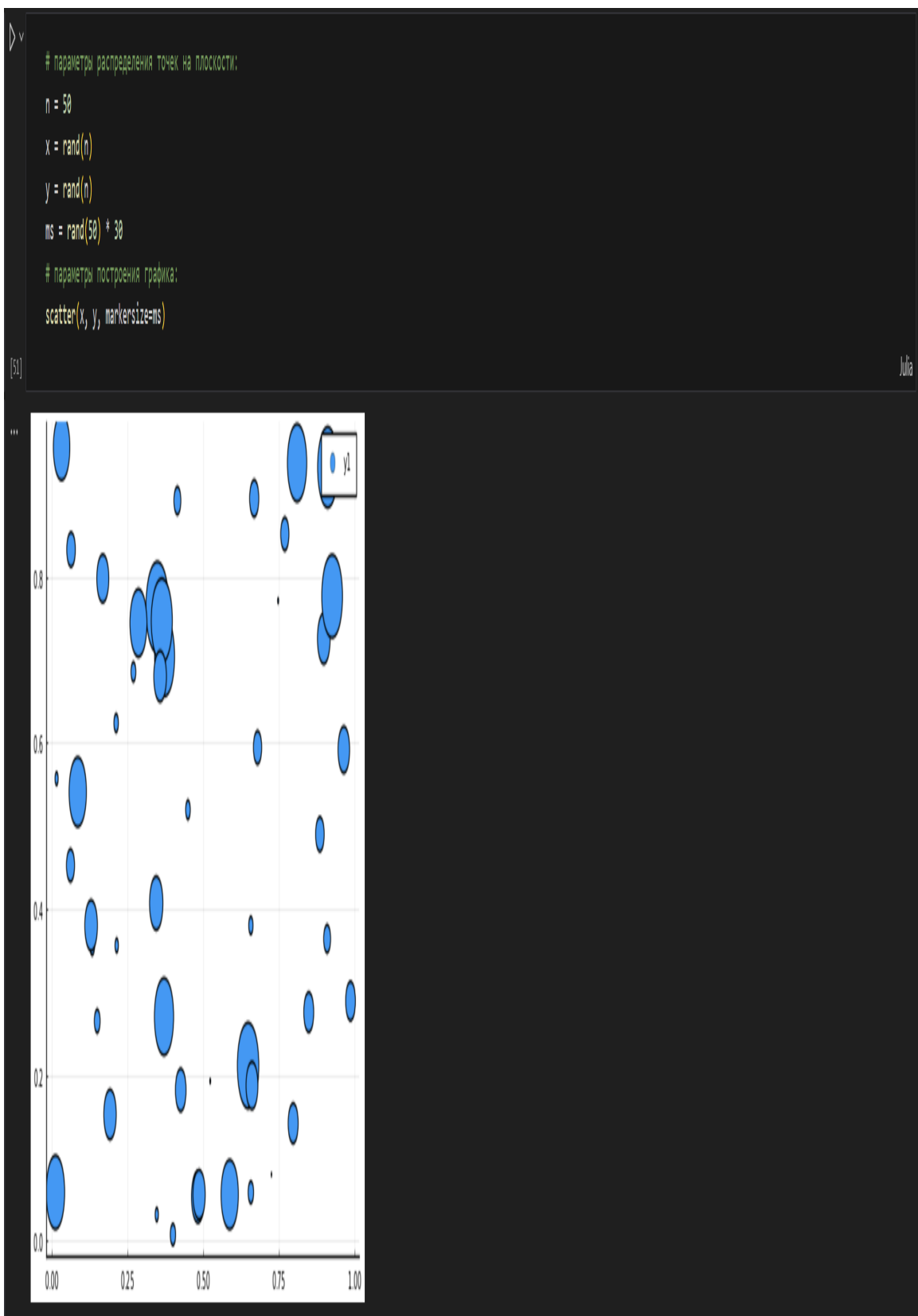


Рисунок 2.12: График пятидесяти случайных значений на плоскости с различными опциями отображения (точечный график с кодированием значения размером точки)

Также можно строить и 3-мерные точечные графики (рис. [fig:013]):

```
# параметры распределения точек в пространстве:
```

```
n = 50
```

```
x = rand(n)
```

```
y = rand(n)
```

```
z = rand(n)
```

```
ms = rand(50) * 30
```

```
# параметры построения графика:
```

```
scatter(x, y, z, markersize=ms,
```

```
        title="3D scatter plot",
```

```
        color="green",
```

```
        xlabel="x",
```

```
        ylabel="y",
```

```
        zlabel="z",
```

```
        label="Случайные значения")
```

[5]:

Julia

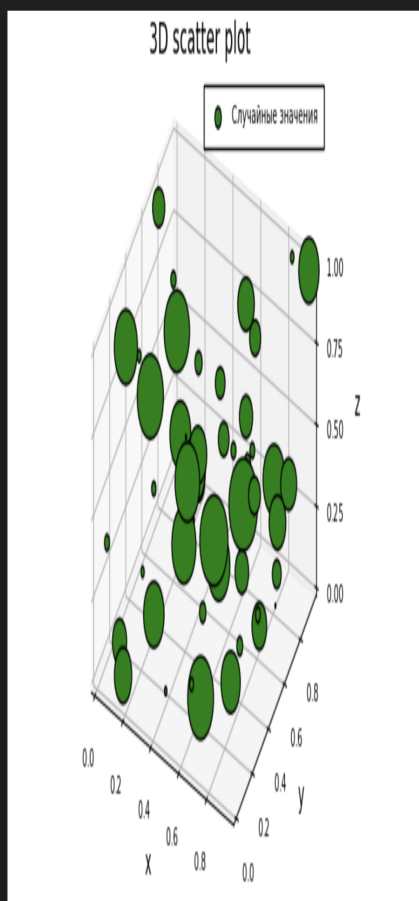


Рисунок 2.13: График пятидесяти случайных значений в пространстве с различными опциями отображения (3-мерный точечный график с кодированием значения размером точки)

2.4 Аппроксимация данных

Аппроксимация — научный метод, состоящий в замене объектов их более простыми аналогами, сходными по своим свойствам.

Для демонстрации зададим искусственно некоторую функцию, в данном случае похожую по поведению на экспоненту (рис. [fig:014]):

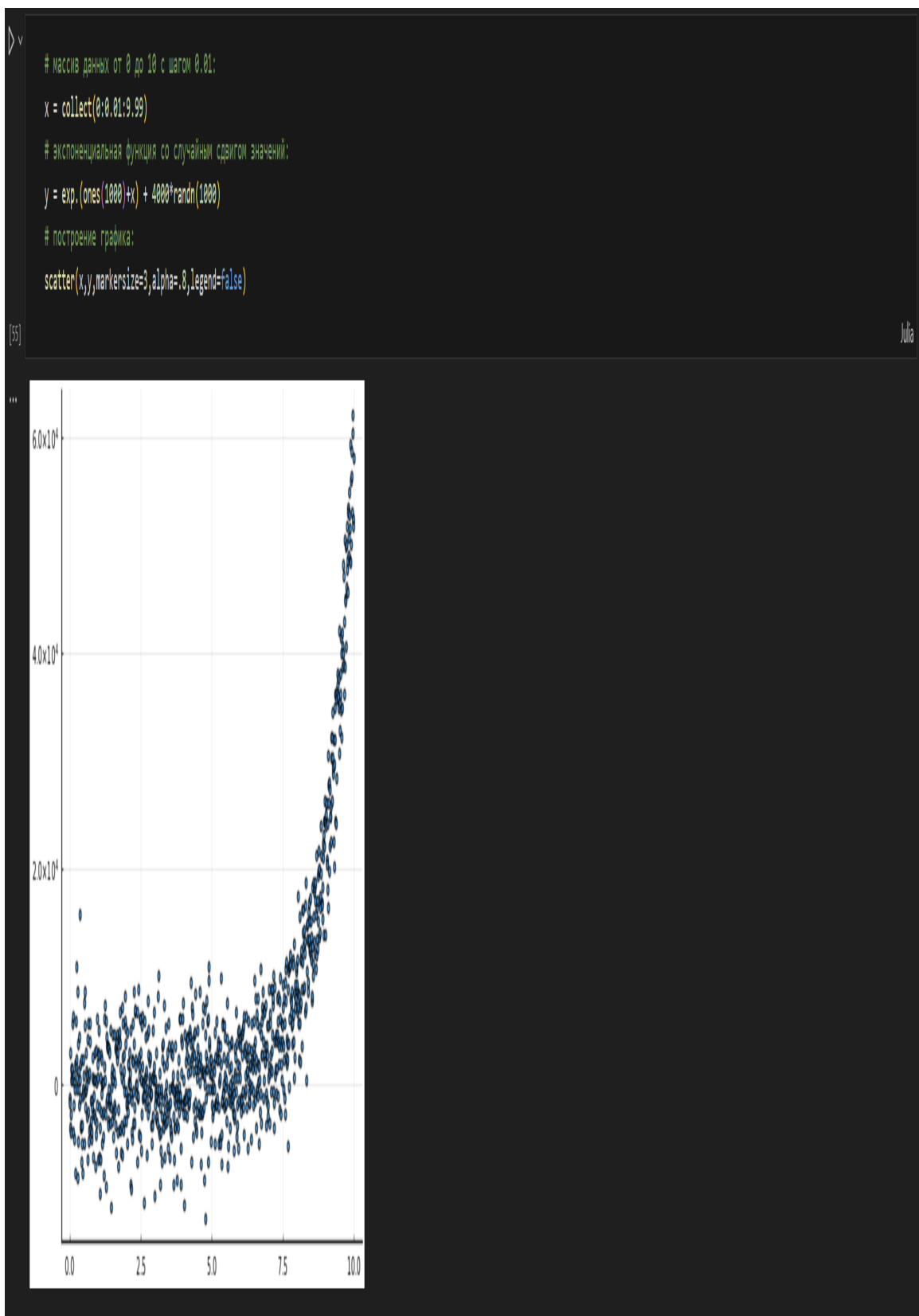


Рисунок 2.14: Пример функции

Аппроксимируем полученную функцию полиномом 5-й степени (рис. [fig:015]):

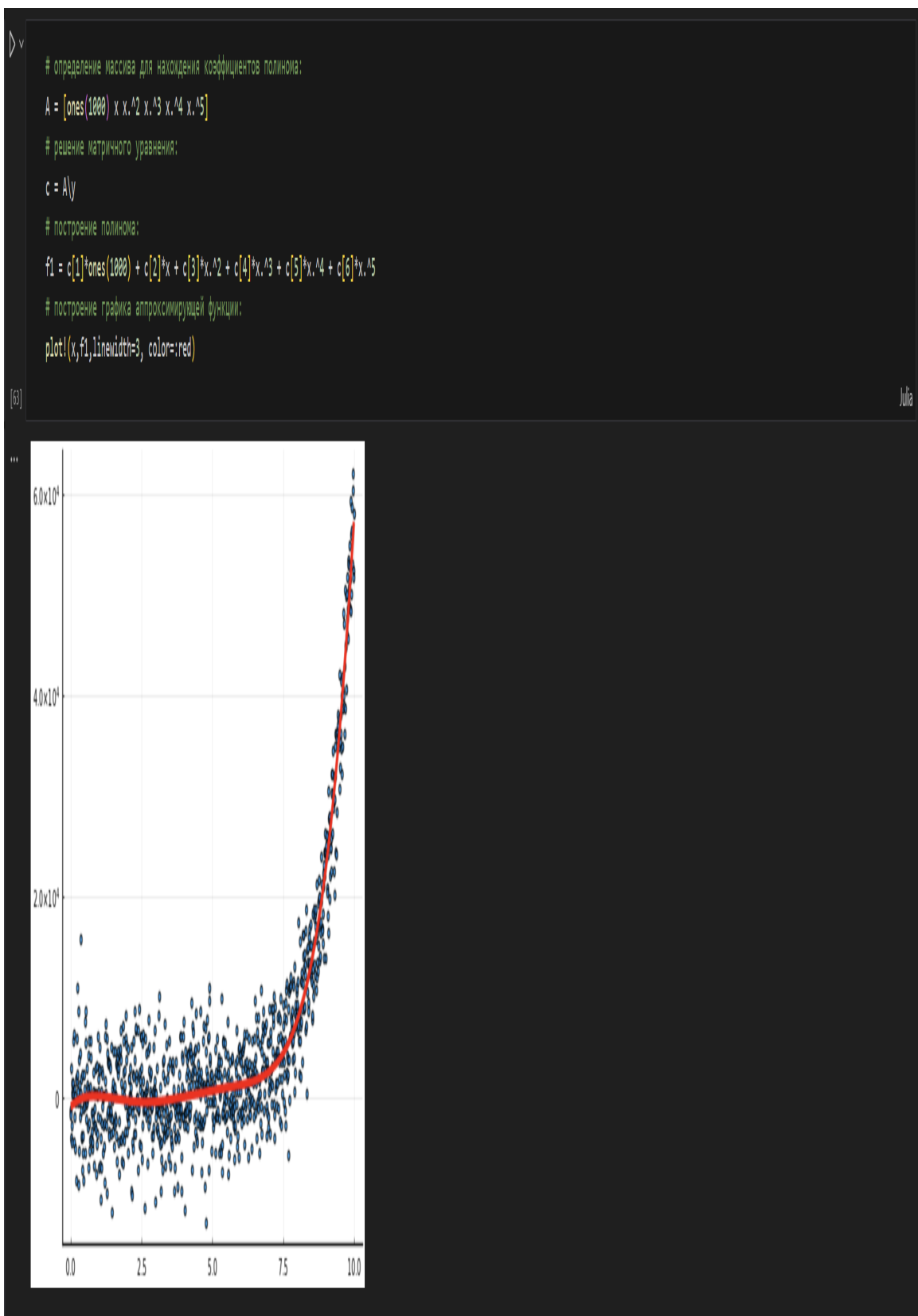


Рисунок 2.15: Пример аппроксимации исходной функции полиномом 5-й степени

2.5 Две оси ординат

Иногда требуется на один график вывести несколько траекторий с существенными отличиями в значениях по оси ординат.

Пример траектории (рис. [fig:016]):

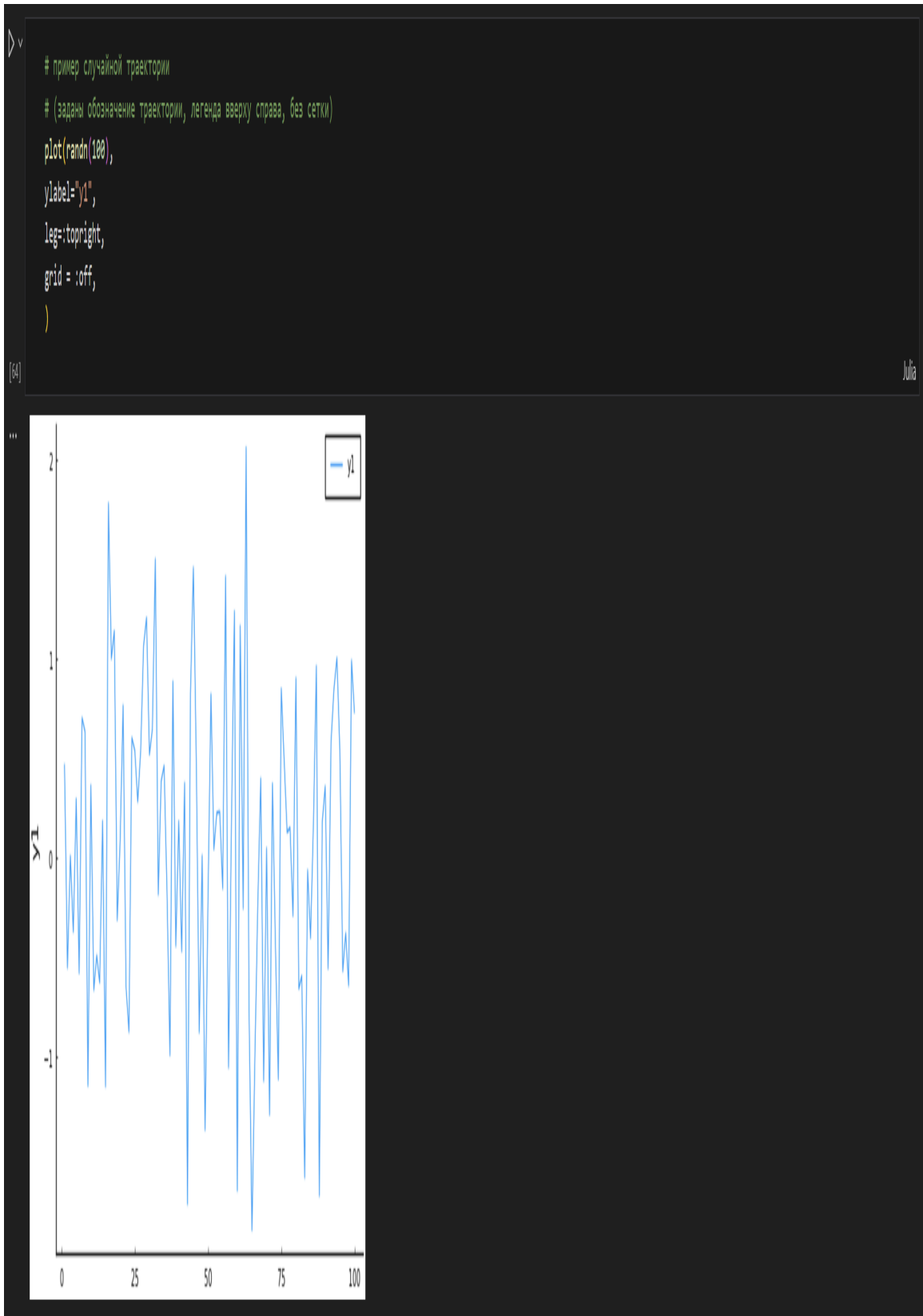


Рисунок 2.16: Пример отдельно построенной траектории

2.6 Полярные координаты

Приведём пример построения графика функции в полярных координатах (рис. [fig:017]):

```
# функция в полярных координатах:
r(θ) = 1 + cos(θ) * sin(θ)^2
# полярная система координат:
θ = range(0, stop=2π, length=50)
# график функции, заданной в полярных координатах:
plot(θ, r.(θ),
proj=:polar,
lins=(0,1.5)
)
```

[60] Julia

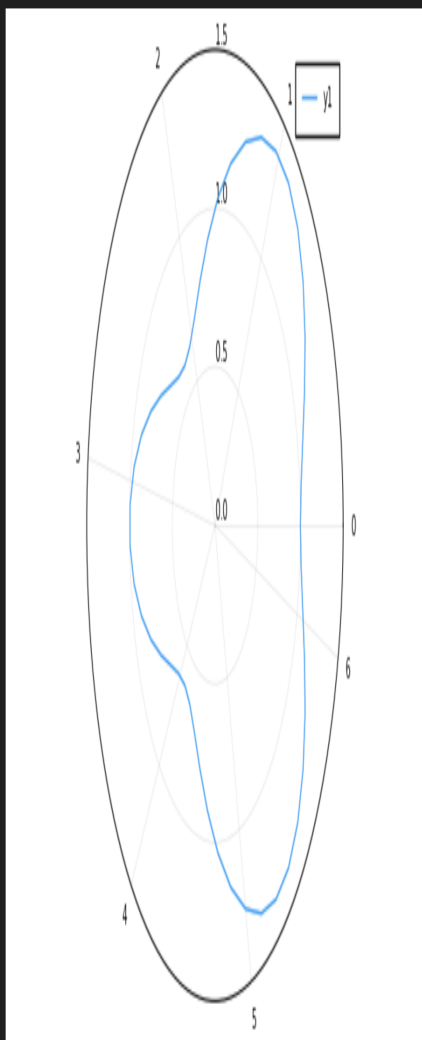


Рисунок 2.17: График функции, заданной в полярных координатах

2.7 Параметрический график

Приведём пример построения графика параметрически заданной кривой на плоскости (рис. [fig:018]):

```
# параметрическое уравнение:  
x1(t) = sin(t)  
y2(t) = sin(2t)  
  
# построение графика:  
plot(x1, y2, 0, 2π, leg=false, fill=(0,orange))
```

[74]

Julia

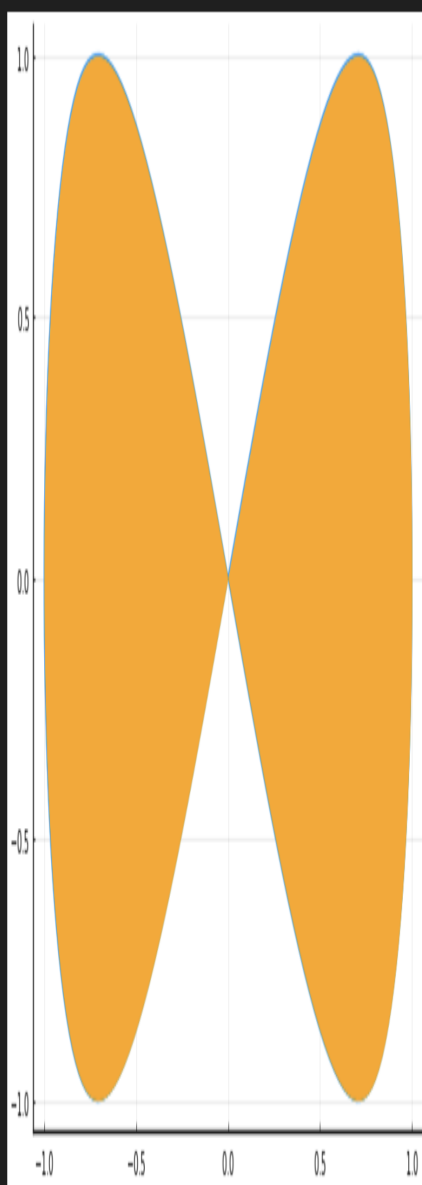


Рисунок 2.18: Параметрический график кривой на плоскости

Далее приведём пример построения графика параметрически заданной кривой в пространстве (рис. [fig:019]):

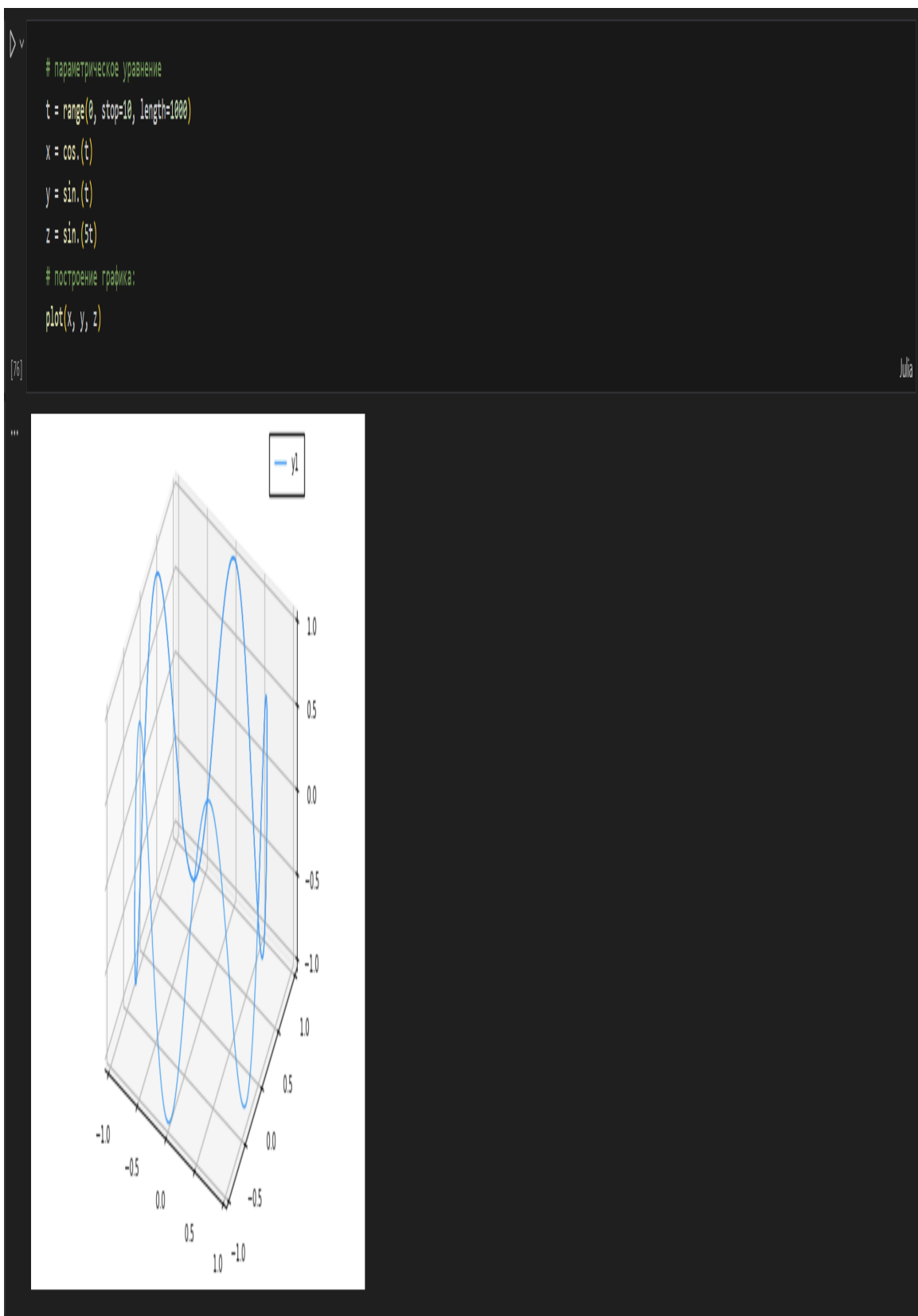


Рисунок 2.19: Параметрический график кривой в пространстве

2.8 График поверхности

Для построения поверхности, заданной уравнением $f(x, y) = x^2 + y^2$, можно воспользоваться функцией `surface()` (рис. [fig:020]):

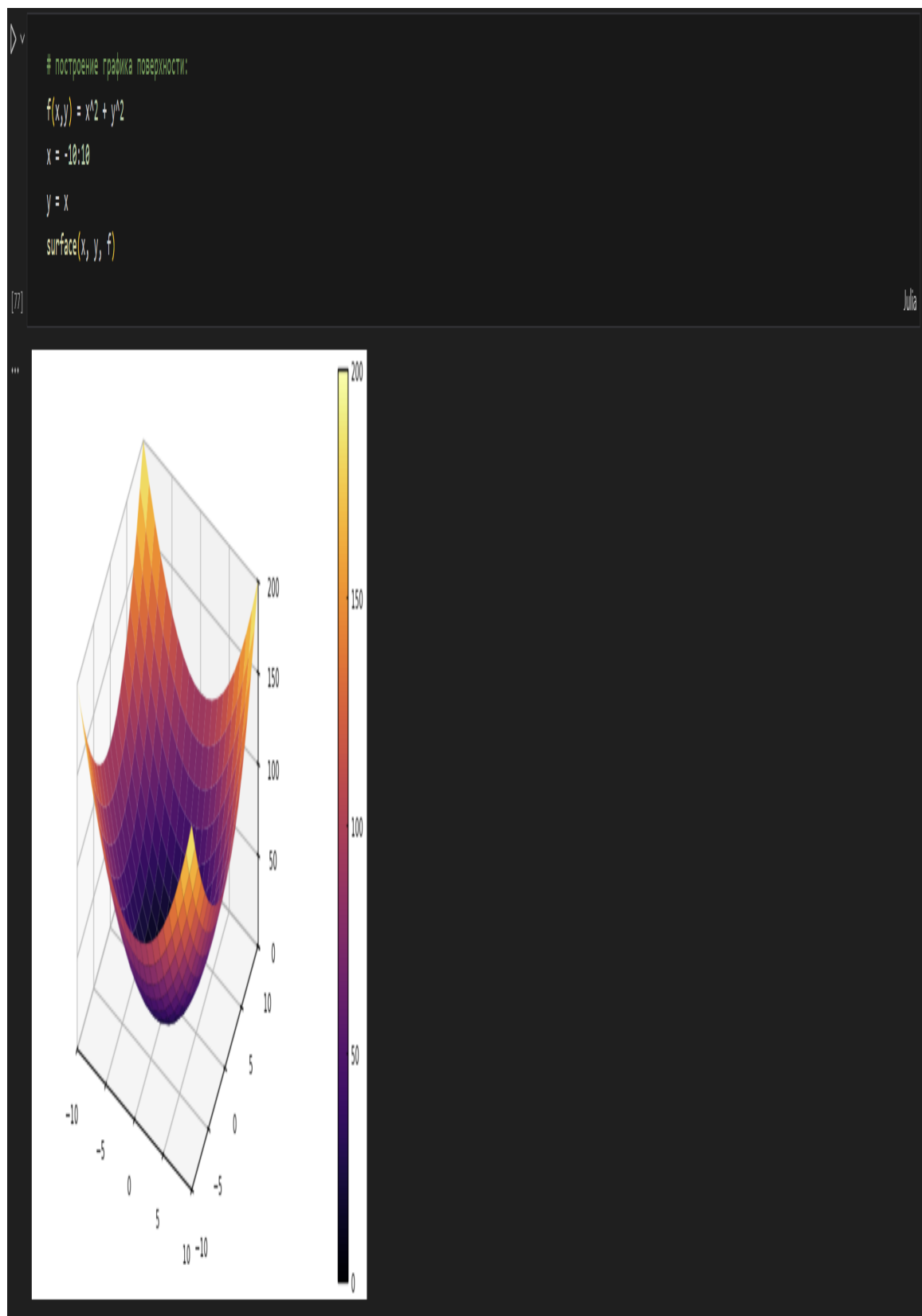


Рисунок 2.20: График поверхности (использована функция `surface()`)

Также можно воспользоваться функцией `plot()` с заданными параметрами (рис. [fig:021]):

```

# построение графика поверхности:
f(x,y) = x^2 + y^2
x = -10:10
y = x
plot(x, y, f,
linetype=:wireframe
)

```

[78]

Julia

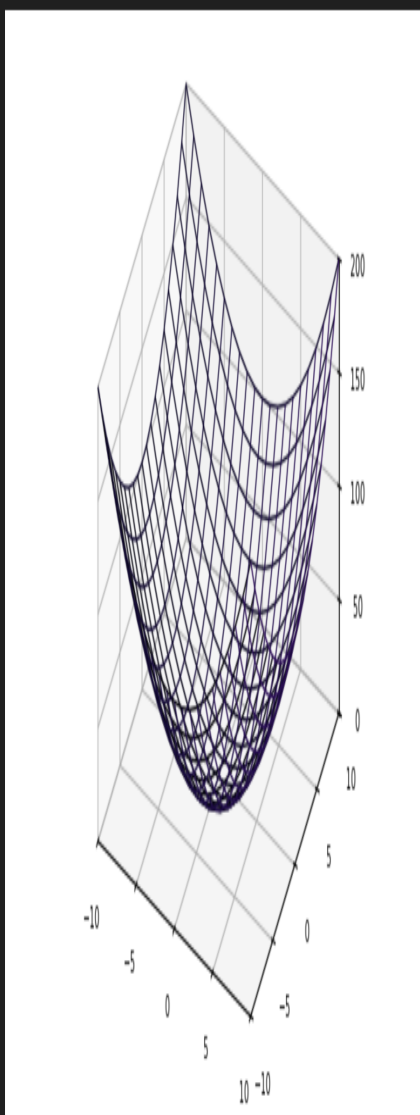


Рисунок 2.21: График поверхности (использована функция `plot()`)

Можно задать параметры сглаживания (рис. [fig:022]):

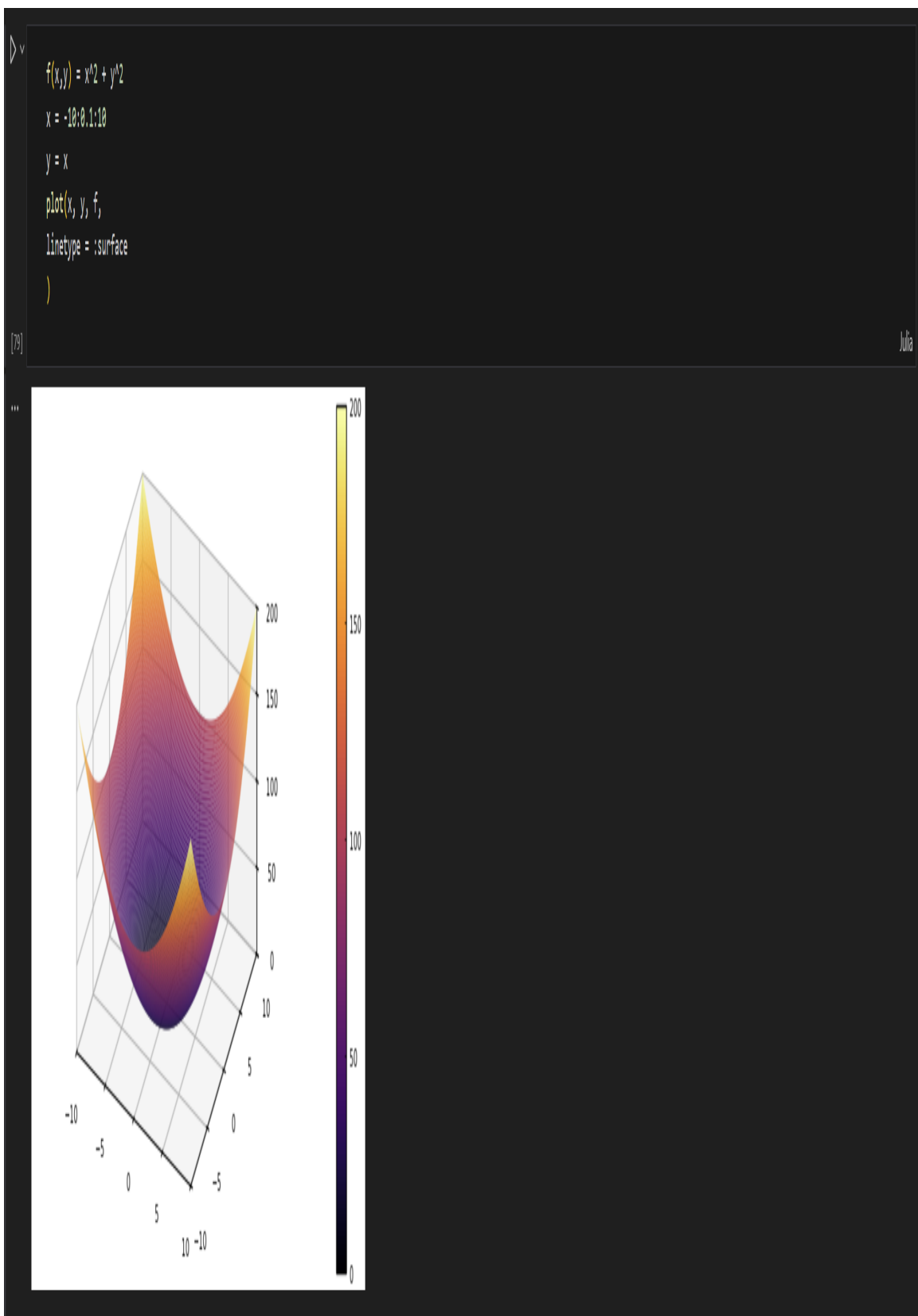


Рисунок 2.22: Сглаженный график поверхности

Можно задать определённый угол зрения (рис. [fig:023]):

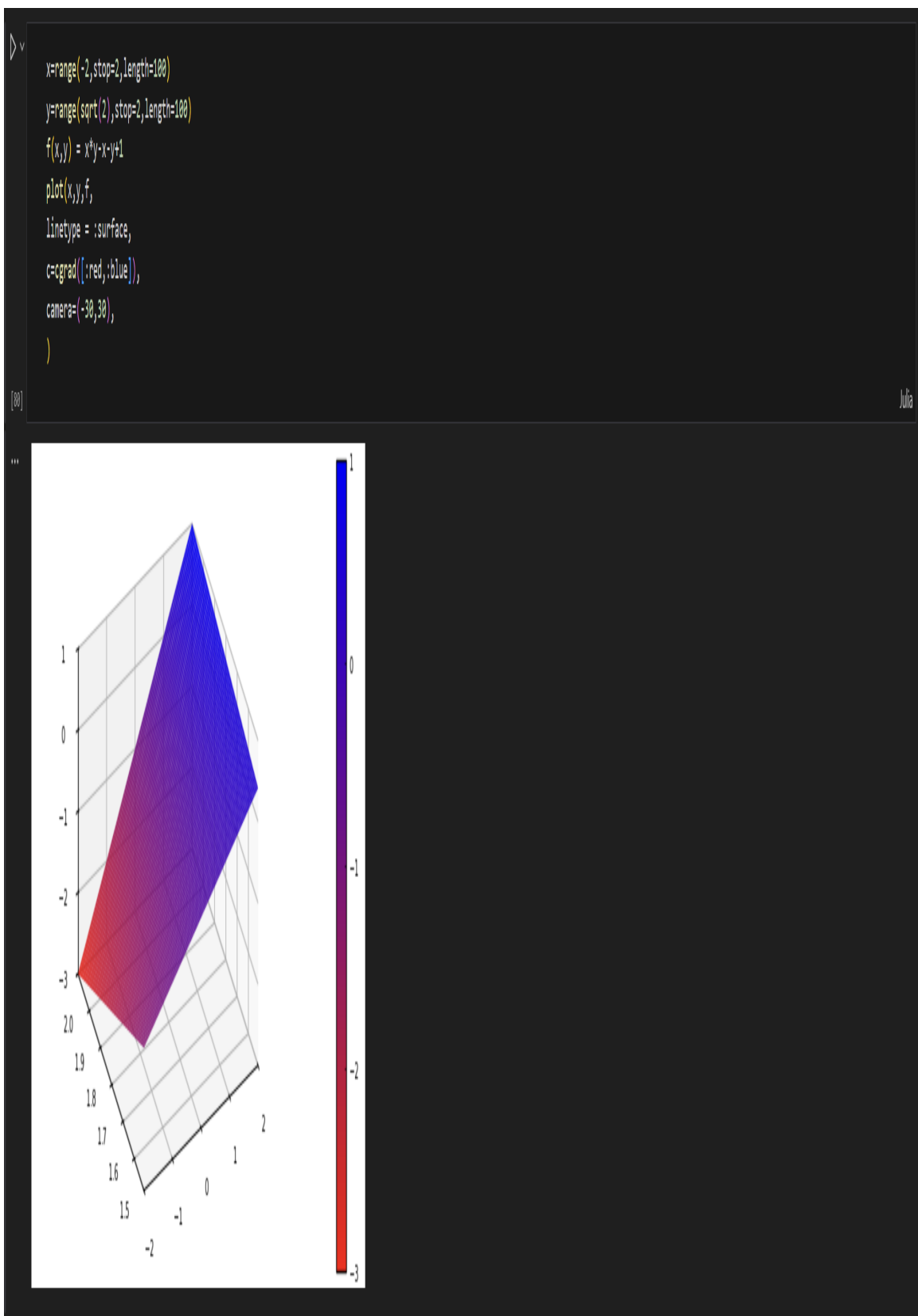


Рисунок 2.23: График поверхности с изменённым углом зрения

2.9 Линии уровня

Линией уровня некоторой функции от двух переменных называется множество точек на координатной плоскости, в которых функция принимает одинаковые значения. Линий уровня бесконечно много, и через каждую точку области определения можно провести линию уровня.

С помощью линий уровня можно определить наибольшее и наименьшее значение исходной функции от двух переменных. Каждая из этих линий соответствует определённому значению высоты.

Поверхности уровня представляют собой непересекающиеся пространственные поверхности.

Рассмотрим поверхность, заданную функцией $g(x, y) = (3x + y^2)|\sin(x) + \cos(y)|$ (рис. [fig:024]):

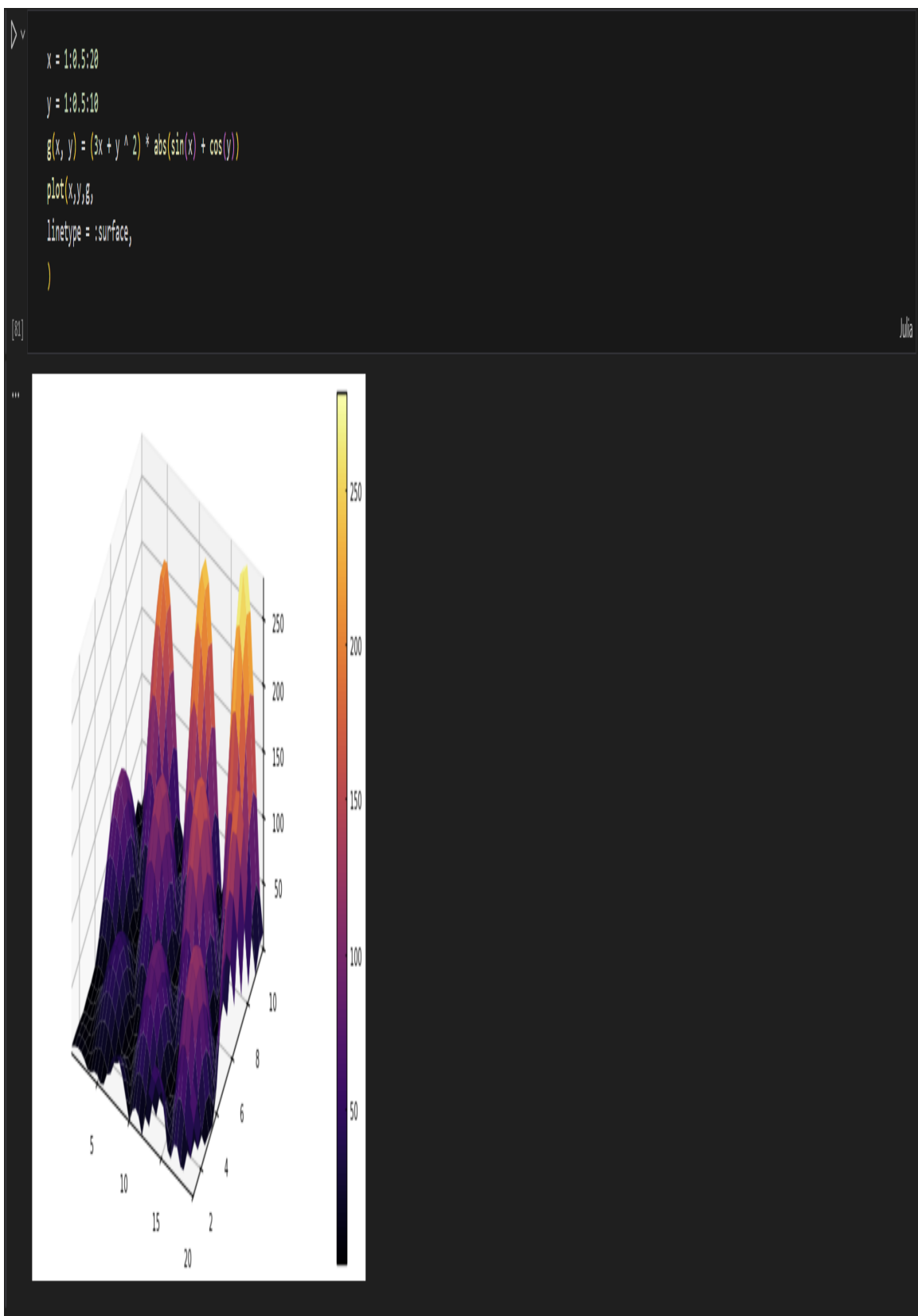


Рисунок 2.24: График поверхности, заданной функцией $g(x, y) = (3x + y^2) |\sin(x) + \cos(y)|$

Линии уровня можно построить, используя проекцию значений исходной функции на плоскость (рис. [fig:025]):

»

```
contour(x, y, g)
```

[83]

Julia

...

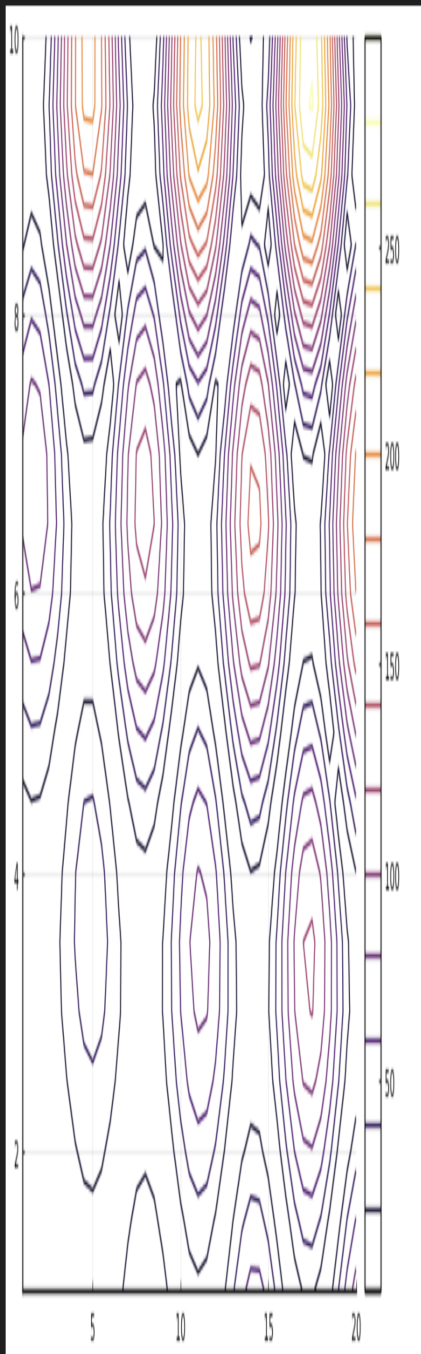


Рисунок 2.25: Линии уровня

Можно дополнительно добавить заливку цветом (рис. [fig:026]):

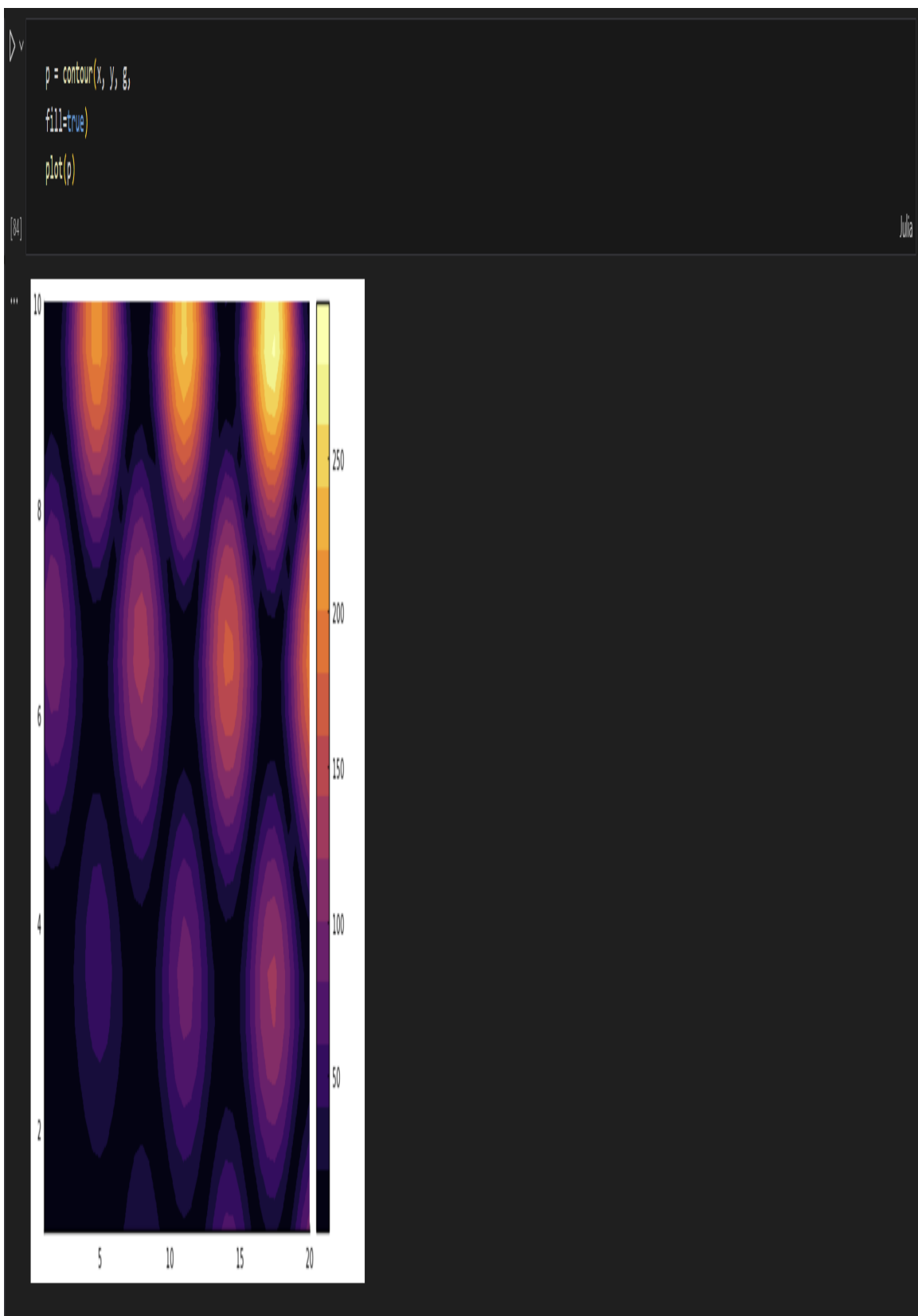


Рисунок 2.26: Линии уровня с заполнением

2.10 Векторные поля

Если каждой точке некоторой области пространства поставлен в соответствие вектор с началом в данной точке, то говорят, что в этой области задано векторное поле.

Векторные поля задают векторными функциями.

Для функции $h(x, y) = x^3 - 3x + y^2$ сначала построим её график (рис. [fig:027]) и линии уровня (рис. [fig:028]):



Рисунок 2.27: График функции $h(x, y) = x^3 - 3x + y^2$

```

> # построение линий уровня:
> contour(X, Y, h)
[97]

```

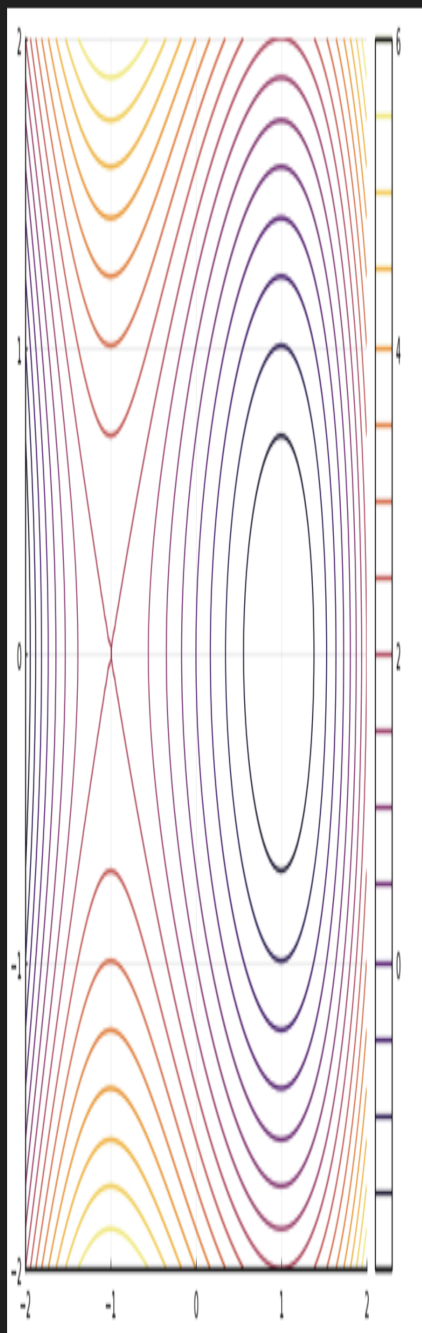


Рисунок 2.28: Линии уровня для функции $h(x, y) = x^3 - 3x + y^2$

2.11 Анимация

Технически анимированное изображение представляет собой несколько наложенных изображений (или построенных в разных точках графиках) в одном файле.

В Julia рекомендуется использовать gif-анимацию в `pyplot()`.

Строим поверхность (рис. [fig:029]):

```
# построение поверхности:
```

```
i = 0
```

```
X = Y = range(-5, stop=5, length=40)
```

```
surface(X, Y, (x,y) -> sin(x+10sin(i))+cos(y))
```

[96]

Julia

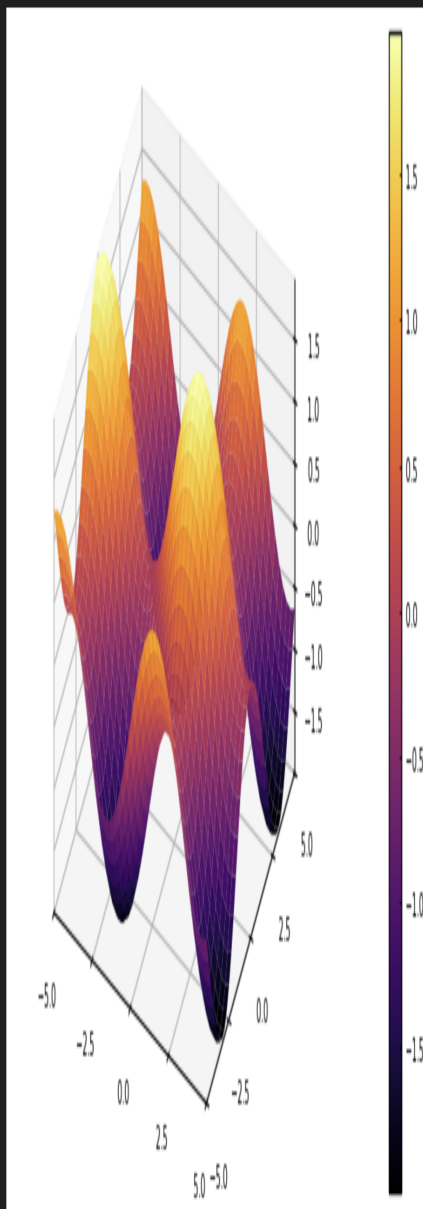


Рисунок 2.29: Статичный график поверхности

Добавляем анимацию (рис. [fig:030]):

```

X = Y = range(-5, stop=5, length=40)
@gif for i in range(0, stop=2π, length=100)
    surface(X, Y, (x,y) -> sin(x+10sin(i))+cos(y))
end

```

[97]

Julia

... Info: Saved animation to C:\Users\durda\AppData\Local\Temp\jl_CpYuv4KaZ.gif

└ @ Plots <C:\Users\durda\julia\packages\Plots\8Zn83\src\animation.jl:156>

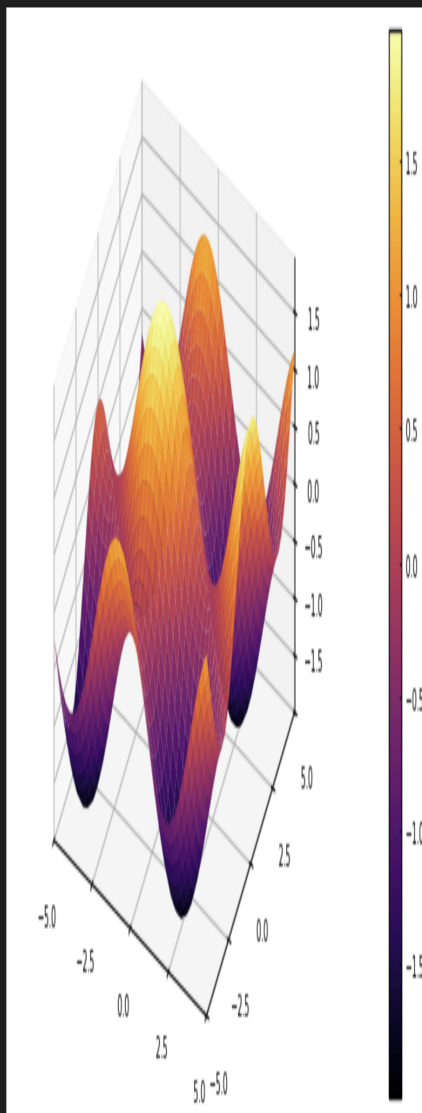


Рисунок 2.30: Анимированный график поверхности

2.12 Гипоциклоида

Гипоциклоида — плоская кривая, образуемая точкой окружности, катящейся по внутренней стороне другой окружности без скольжения.

Построим большую окружность (рис. [fig:031] - рис. [fig:032]):


```
# радиус малой окружности:
r1 = 1

# коэффициент для построения большой окружности:
k_1 = 3

# число отсчётов:
n1 = 100

[8] ✓ 0.1s Julia

... 100

# массив значений угла  $\theta$ :
# theta from 0 to  $2\pi$  ( + a little extra)
 $\theta = \text{collect}(0:2\pi/100:2\pi+2\pi/100)$ 

# массивы значений координат:
X = r1*k_1*cos.( $\theta$ )
Y = r1*k_1*sin.( $\theta$ )

[9] ✓ 0.5s Julia

... 101-element Vector{Float64}:
 0.0
 0.18837155858794014
 0.3759997006929128
 0.5621439437571739
 0.746096614945643
 0.9270509831240421
 1.104373658054034
 1.2773378746952182
 1.445261022305146
 1.60748030493699
 ⋮
-1.445261022305146
-1.2773378746952166
-1.1043736580540335
-0.9270509831240404
-0.746096614945634
-0.562143943757174
-0.37599970069291133
-0.1883715585879398
 1.929747179611964e-15
```

Рисунок 2.31: Построение большой окружности гипоциклоиды

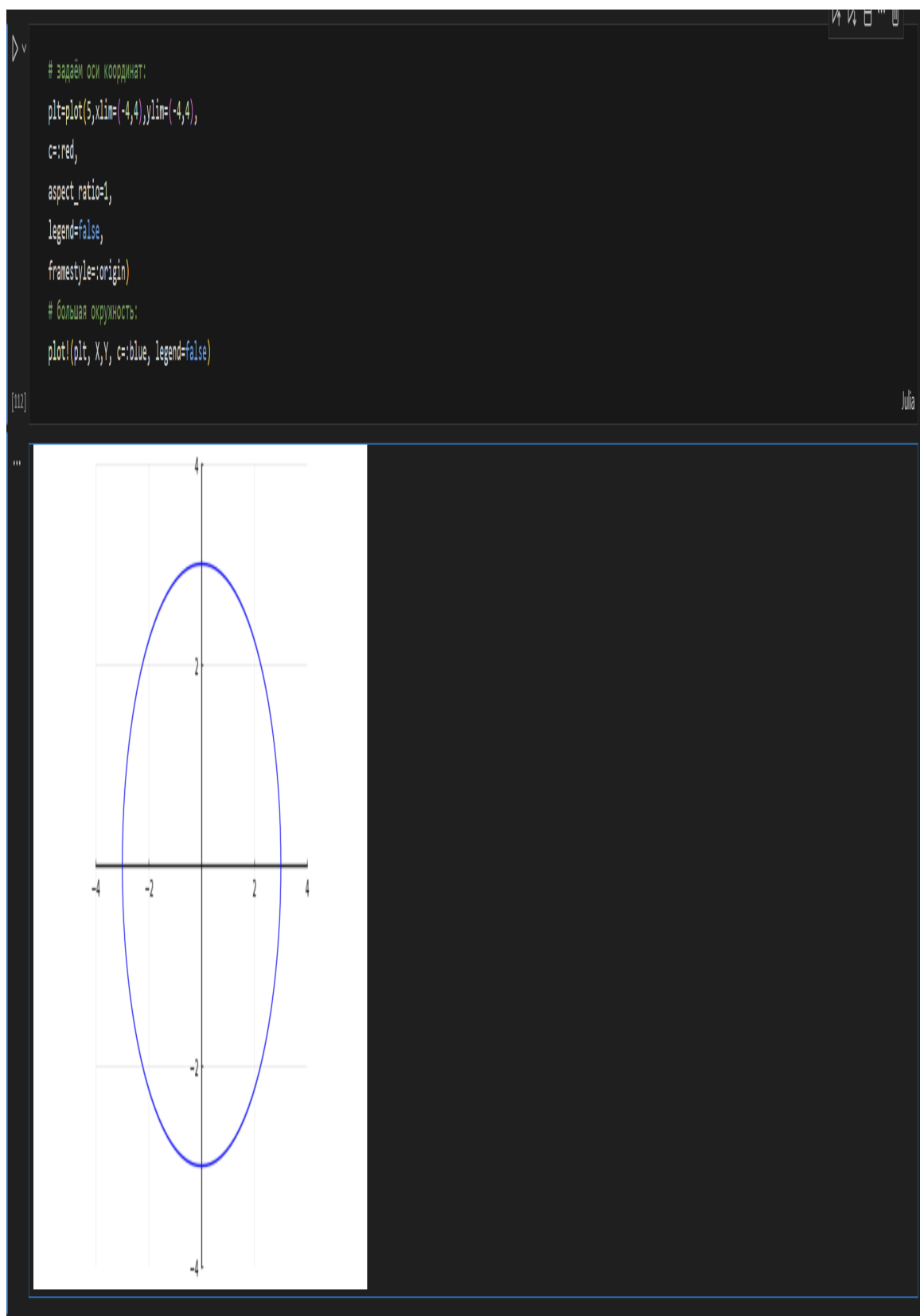


Рисунок 2.32: Большая окружность гипоциклоиды

Для частичного построения гипоциклоиды будем менять параметр t (рис. [fig:033]):

```

i = 50
t = 0[1:i]
# гипоциклоида:
x = r1*(k_1-1)*cos.(t) + r1*cos.((k_1-1)*t)
y = r1*(k_1-1)*sin.(t) - r1*sin.((k_1-1)*t)
plot!(x,y, c=:red)

```

[114]

Julia

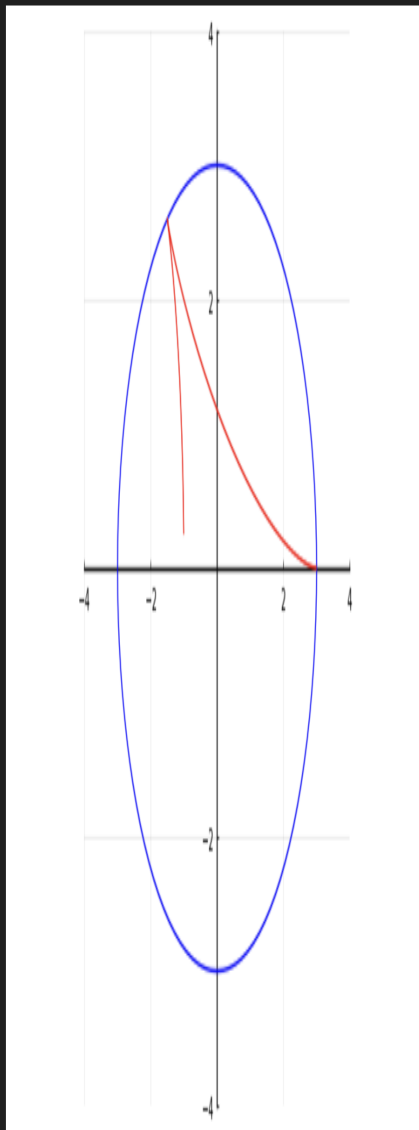


Рисунок 2.33: Половина пути гипоциклоиды

Добавляем малую окружность гипоциклоиды (рис. [fig:034]):

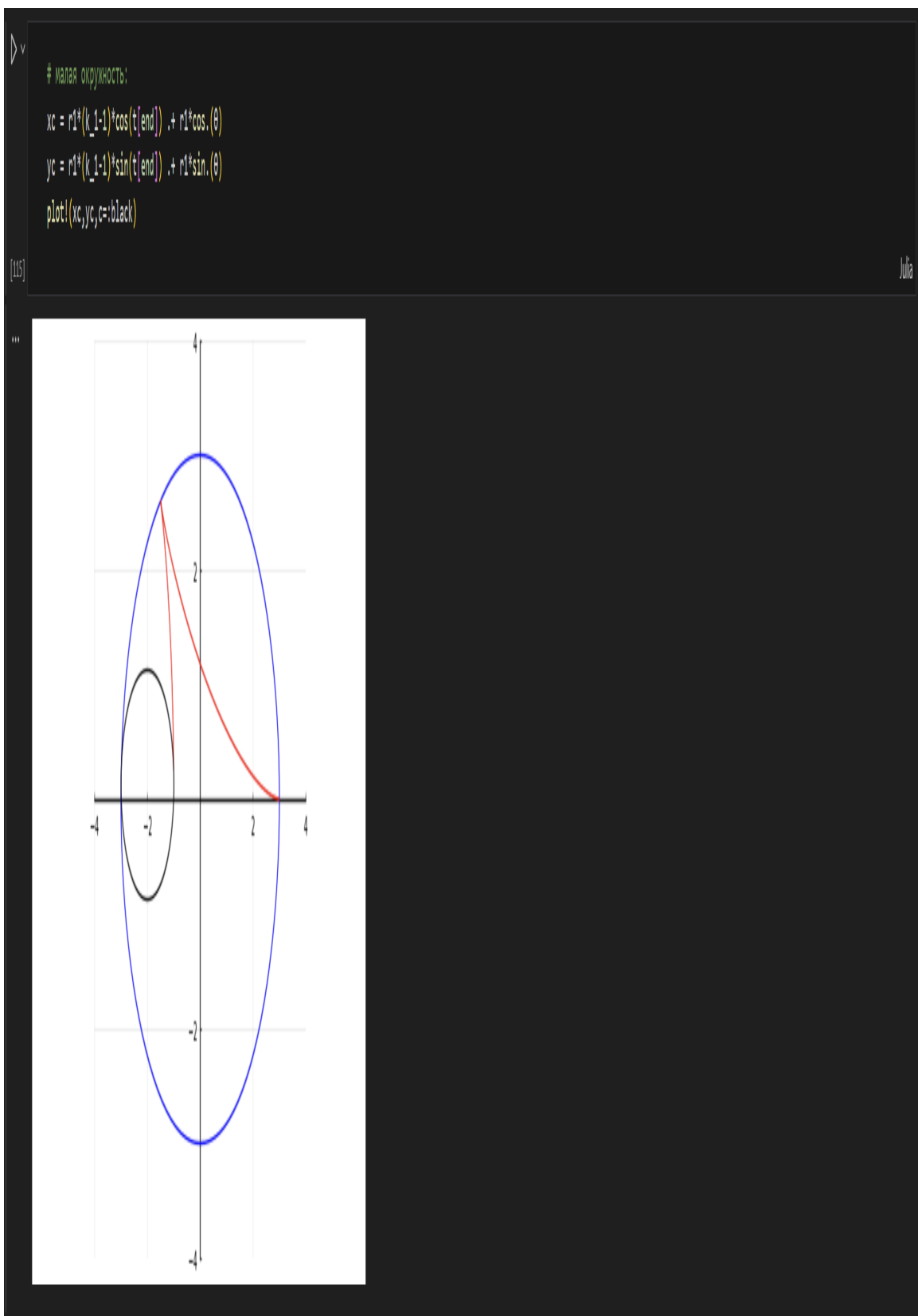


Рисунок 2.34: Малая окружность гипоциклоиды

Добавим радиус для малой окружности (рис. [fig:035]):

```
> v
# радиус малой окружности:
x1 = transpose([r1*(k_1-1)*cos(t[end]) x[end]])
y1 = transpose([r1*(k_1-1)*sin(t[end]) y[end]])
plot!(x1,y1,markershape=:circle,markersize=4,c=:black)
scatter!([x[end]],[y[end]],c=:red, markerstrokecolor=:red)

[116] Julia
```

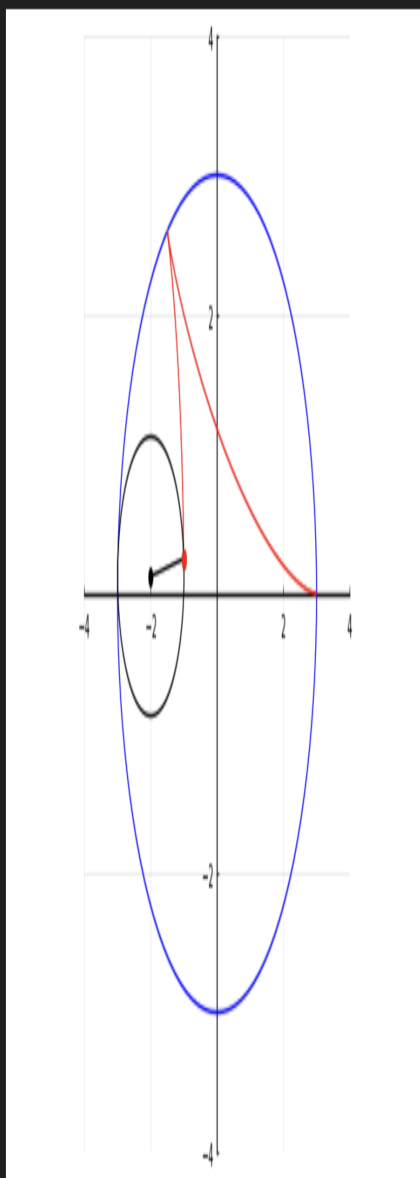


Рисунок 2.35: Малая окружность гипоциклоиды с добавлением радиуса

2.13 Errorbars

В исследованиях часто требуется изобразить графики погрешностей измерения.

Построим график исходных значений (рис. [fig:036] - рис. [fig:037]):

```
# подключение пакета Statistics:
import Pkg
Pkg.add("Statistics")
using Statistics

[121] Julia

... Resolving package versions...
Updating `C:\Users\durda\.julia\environments\v1.12\Project.toml`
[10745b16] + Statistics v1.11.1
Manifest No packages added to or removed from `C:\Users\durda\.julia\environments\v1.12\Manifest.toml`

▷ v
sds = [1, 1/2, 1/4, 1/8, 1/16, 1/32]

[122] Julia

... 6-element Vector{Float64}:
 1.0
 0.5
 0.25
 0.125
 0.0625
 0.03125

▷ v
n = 10
y = [mean(sd*randn(n)) for sd in sds]
errs = 1.96 * sds / sqrt(n)

[123] Julia

... 6-element Vector{Float64}:
 0.6190064213930023
 0.3099032106965012
 0.1549516053482506
 0.0774758026741253
 0.03873790133706265
 0.019368950668531323
```

Рисунок 2.36: Зададим исходные значения

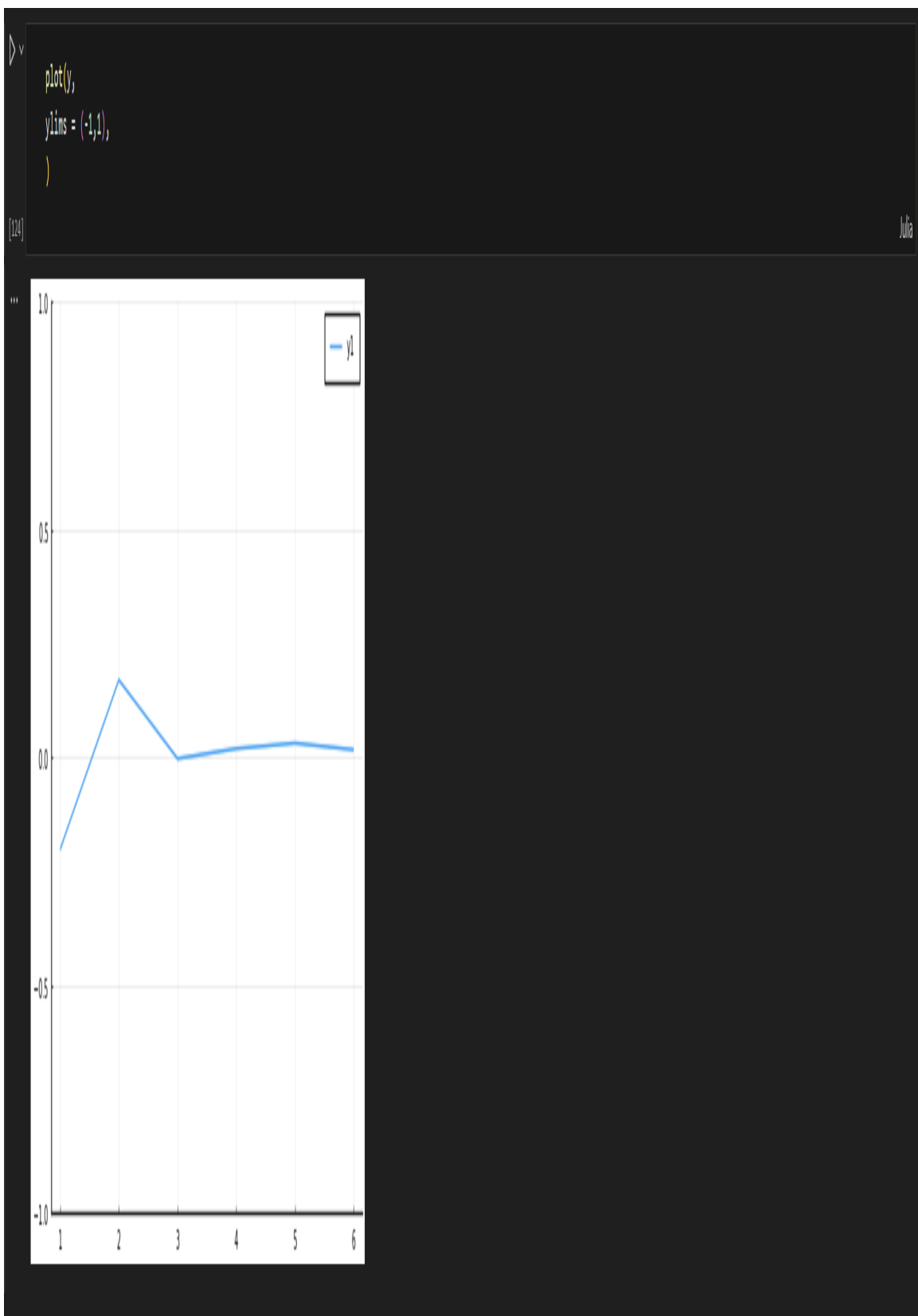


Рисунок 2.37: График исходных значений

Построим график отклонений от исходных значений (рис. [fig:038]):

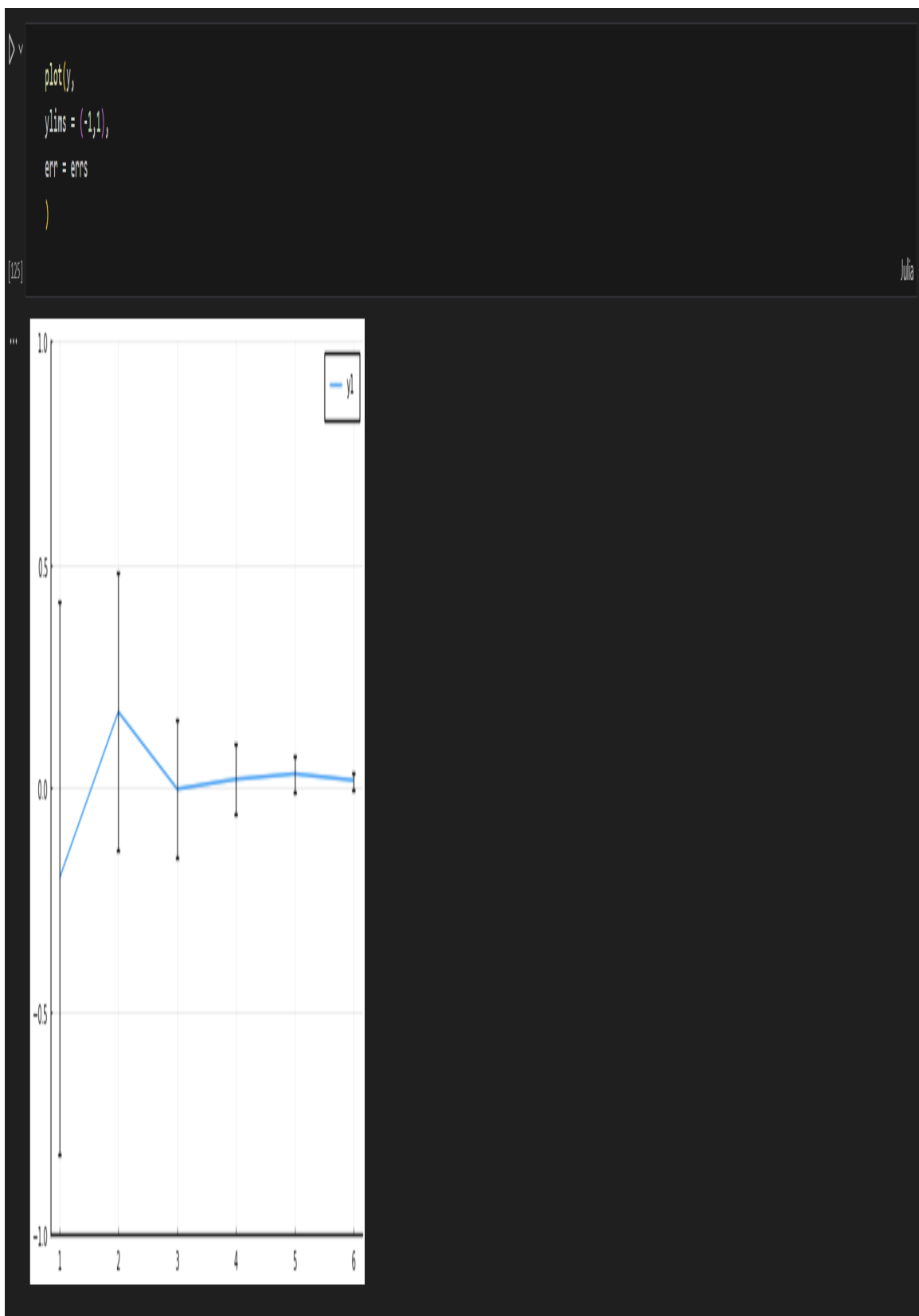


Рисунок 2.38: График исходных значений с отклонениями

Повернём график (рис. [fig:039]):

```
> plot(y, 1:length(y),  
      xerr = errs,  
      marker = stroke(3,:orange)  
      )
```

[126]

Julia

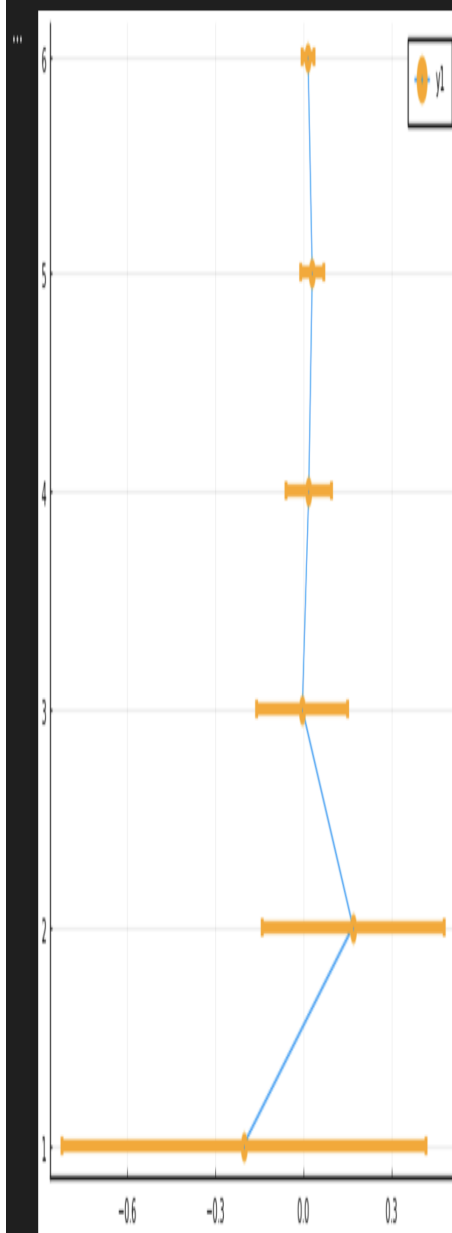


Рисунок 2.39: Поворот графика

Заполним область цветом (рис. [fig:040]):

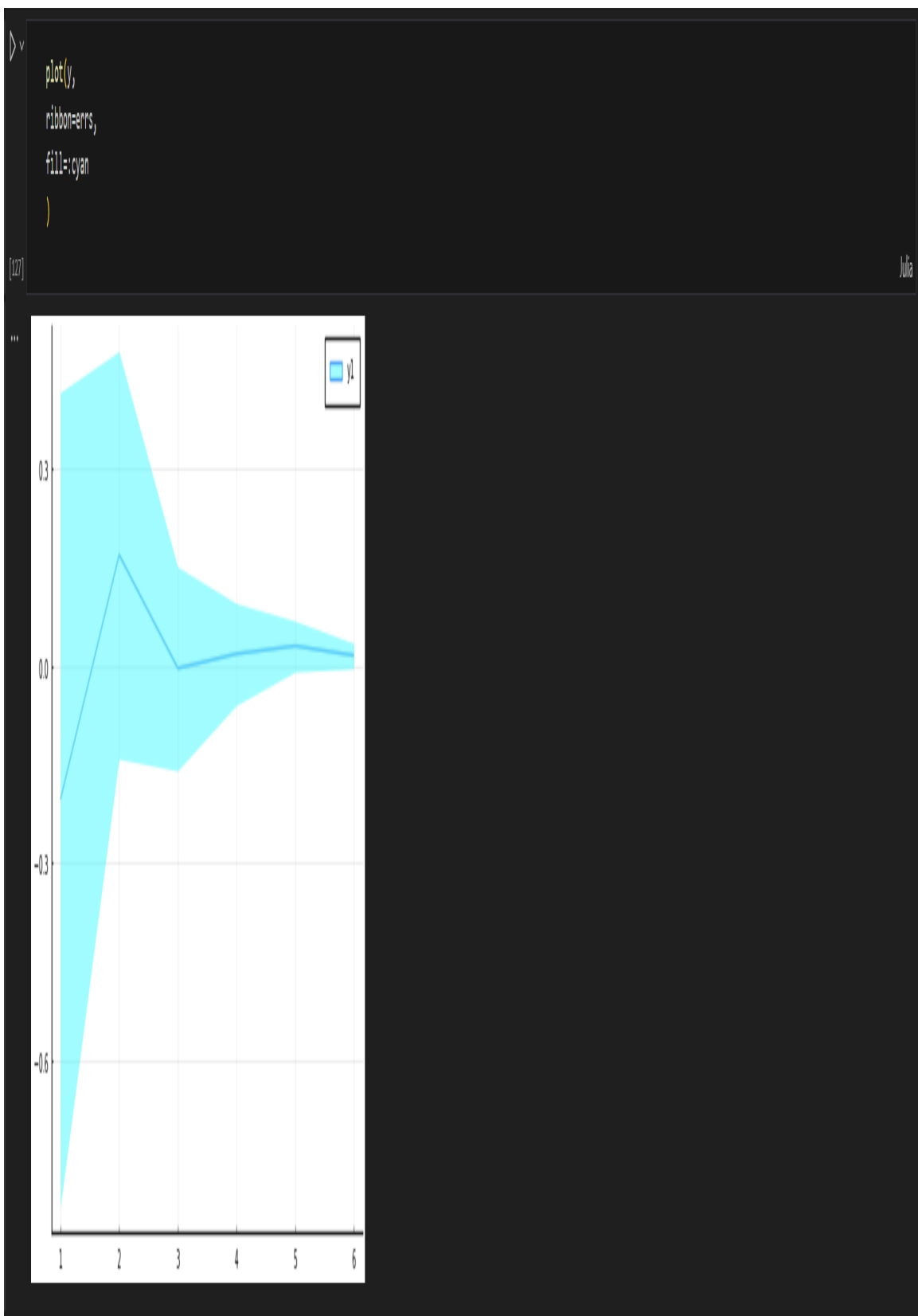


Рисунок 2.40: Заполнение цветом

Можно построить график ошибок по двум осям (рис. [fig:041]):

```

n = 10
x = [(rand()*1) .* randn(n) .+ 2i for i in 1:5]
y = [(rand()*1) .* randn(n) .+ i for i in 1:5]
f(v) = 1.96std(v) / sqrt(n)
xerr = map(f, x)
yerr = map(f, y)
x = map(mean, x)
y = map(mean, y)
plot(x, y,
xerr = xerr,
yerr = yerr,
marker = stroke(2, :orange)
)

```

[128] Julia

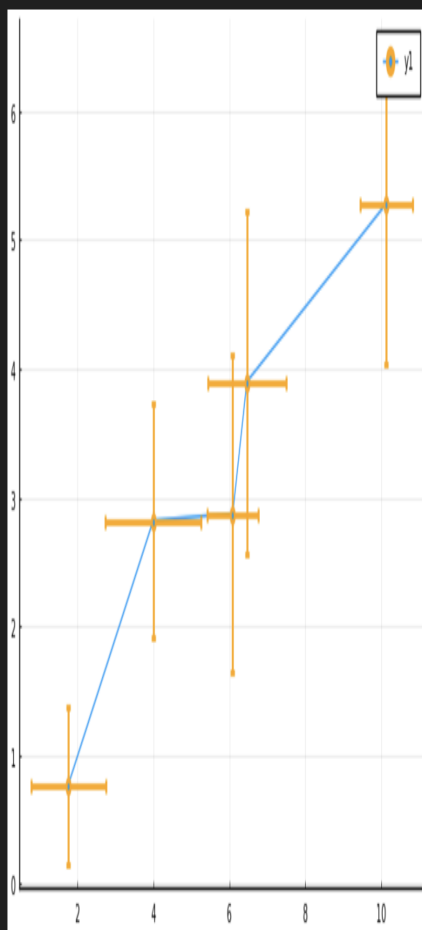


Рисунок 2.41: График ошибок по двум осям

Можно построить график асимметричных ошибок по двум осям (рис. [fig:042]):

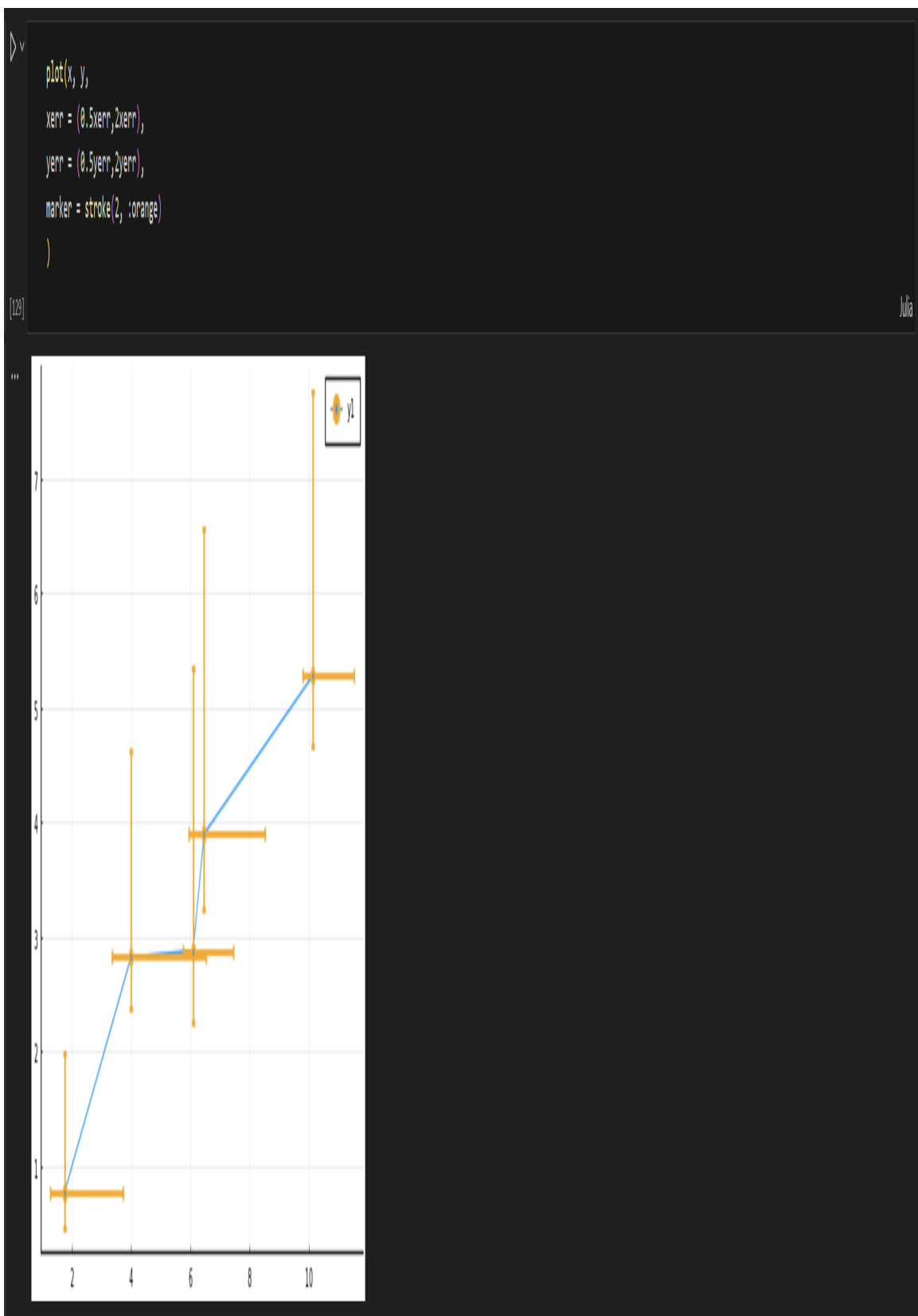


Рисунок 2.42: График асимметричных ошибок по двум осям

2.14 Использование пакета Distributions

Строим гистограмму (рис. [fig:043]):

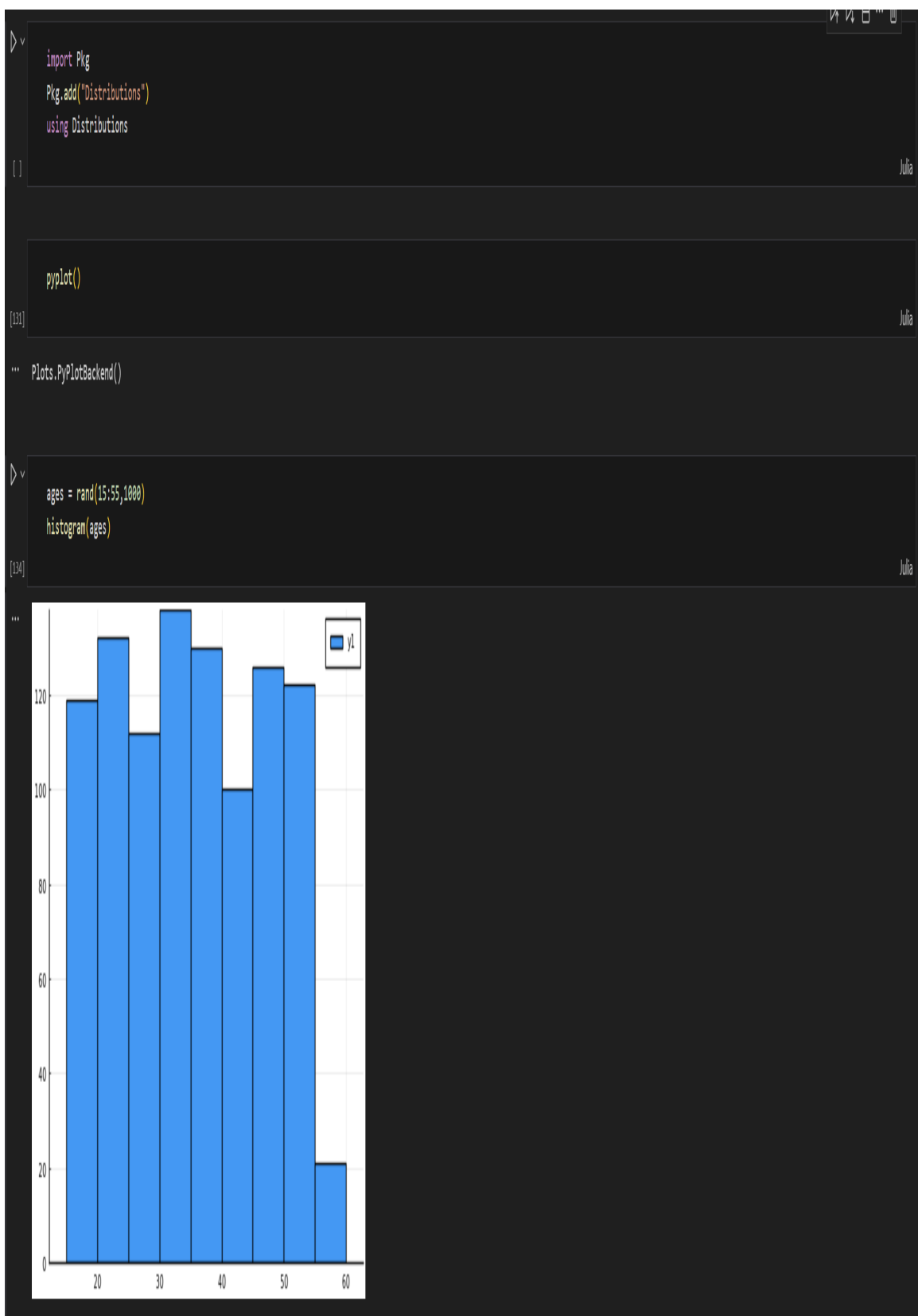


Рисунок 2.43: Гистограмма, построенная по массиву случайных чисел

Задаём нормальное распределение и строим гистограмму (рис. [fig:044]):


```

d=Normal(35.0,10.0)
ages = rand(d,1000)
histogram(
    ages,
    label="Распределение по возрастам (года)",
    xlabel = "Возраст (лет)",
    ylabel= "Количество"
)

```

[135]

Julia

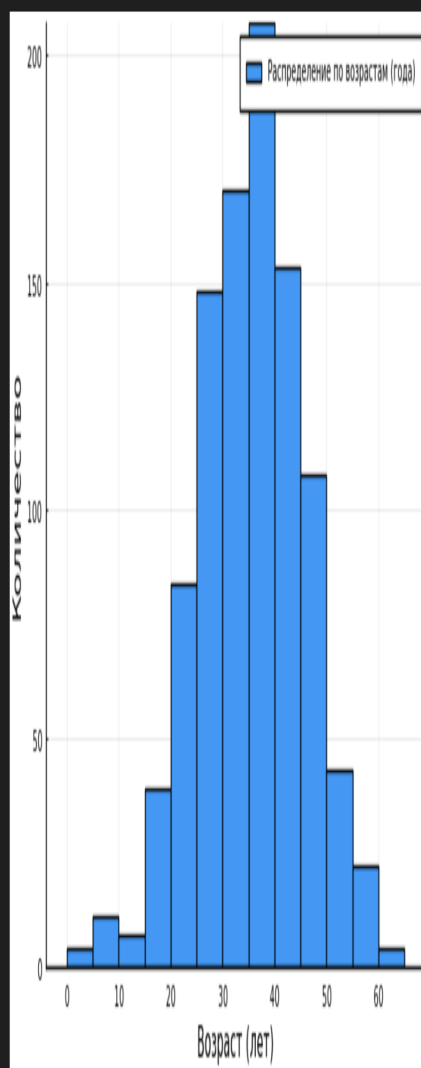


Рисунок 2.44: Гистограмма нормального распределения

2.15 Подграфики

Определим макет расположения графиков. Команда `layout` принимает кортеж `layout = (N, M)`, который строит сетку графиков $N \times M$. Например, если задать `layout = (4, 1)` на графике четыре серии, то получим четыре ряда графиков (рис. [fig:041]):

```
# подгружаем pyplot():
pyplot()

# построение серии графиков:
x=range(-2,2,length=10)
y = rand(10,4)
plot(x,y,
      layout=(4,1)
    )
```

[130]

Julia

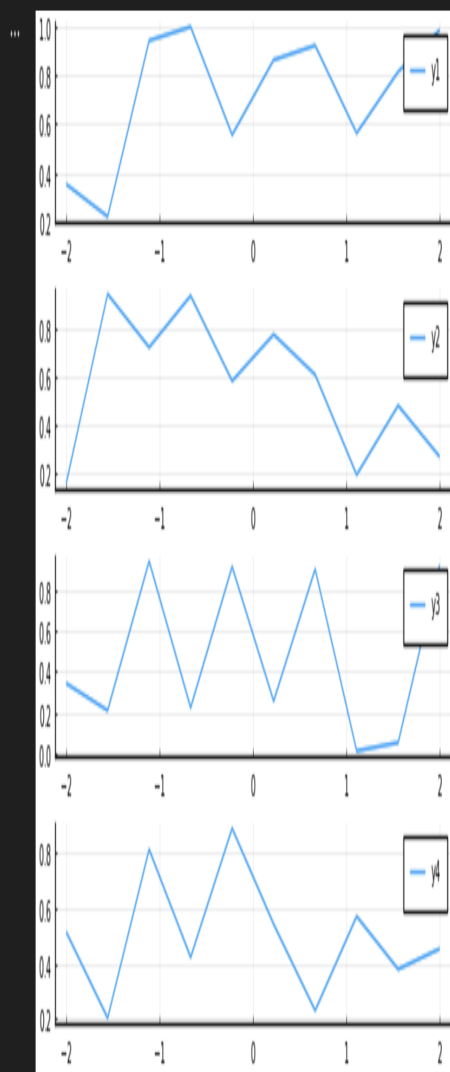


Рисунок 2.45: Серия из 4-х графиков в ряд

Для автоматического вычисления сетки необходимо передать layout целое число
(рис. [fig:046]):

```
plot(x,y,  
      layout=4  
    )
```

[140]

Julia

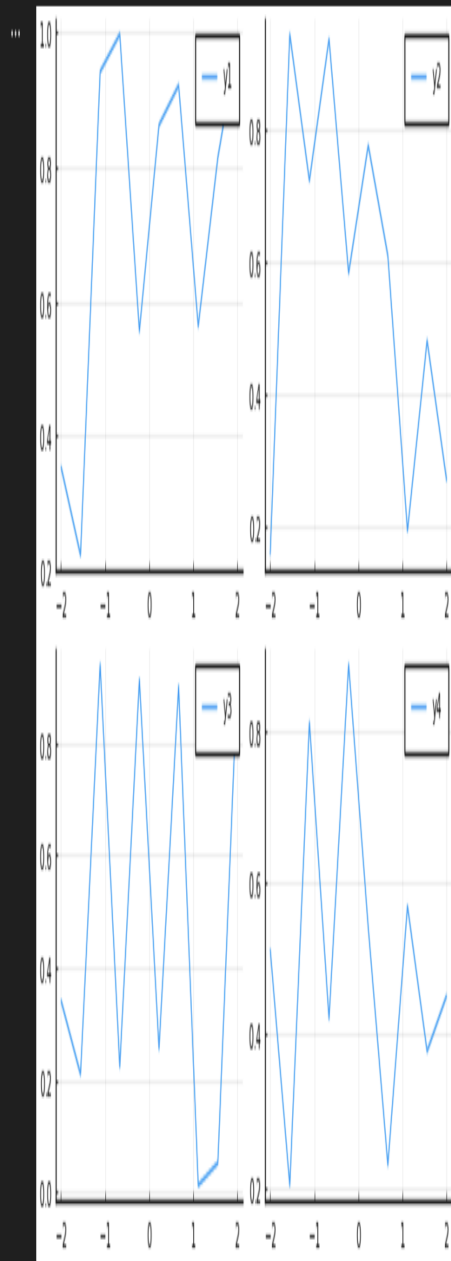


Рисунок 2.46: Серия из 4-х графиков в сетке

Аргумент `heights` принимает в качестве входных данных массив с долями желаемых высот. Если в сумме дроби не составляют 1,0, то некоторые подзаголовки могут отображаться неправильно (рис. [fig:047]).

```
plot(x,y,  
size=(600,300),  
layout = grid(4,1,heights=[0.2,0.25,0.4,0.15])  
)
```

[14] ✓ 0.8s Julia

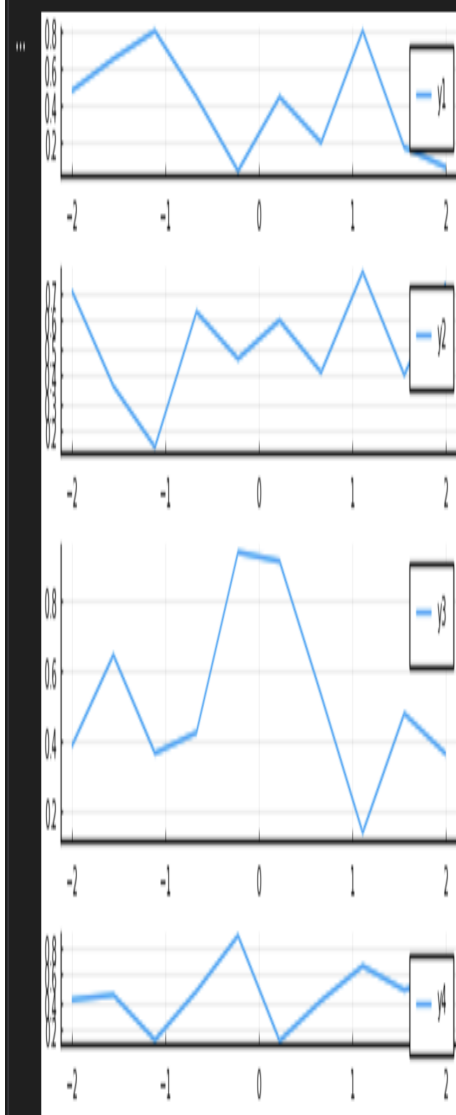


Рисунок 2.47: Серия из 4-х графиков разной высоты в ряд

Можно сгенерировать отдельные графики и объединить их в один, например, в сетке 2×2 (рис. [fig:048] - рис. [fig:049]):


```

# график в виде линий:
p1 = plot(x,y)

# график в виде точек:
p2 = scatter(x,y)

# график в виде линий с оформлением:
p3 = plot(x,y[:,1:2],xlabel="labelled plot of two
↳ columns",lw=2,title="wide lines")

# 4 гистограммы:
p4 = histogram(x,y)

plot(
p1,p2,p3,p4,
layout=(2,2),
legend=false,
size=(800,600),
background_color = :ivory
)

```

[142]

Julia

Рисунок 2.48: Объединение нескольких графиков в одной сетке

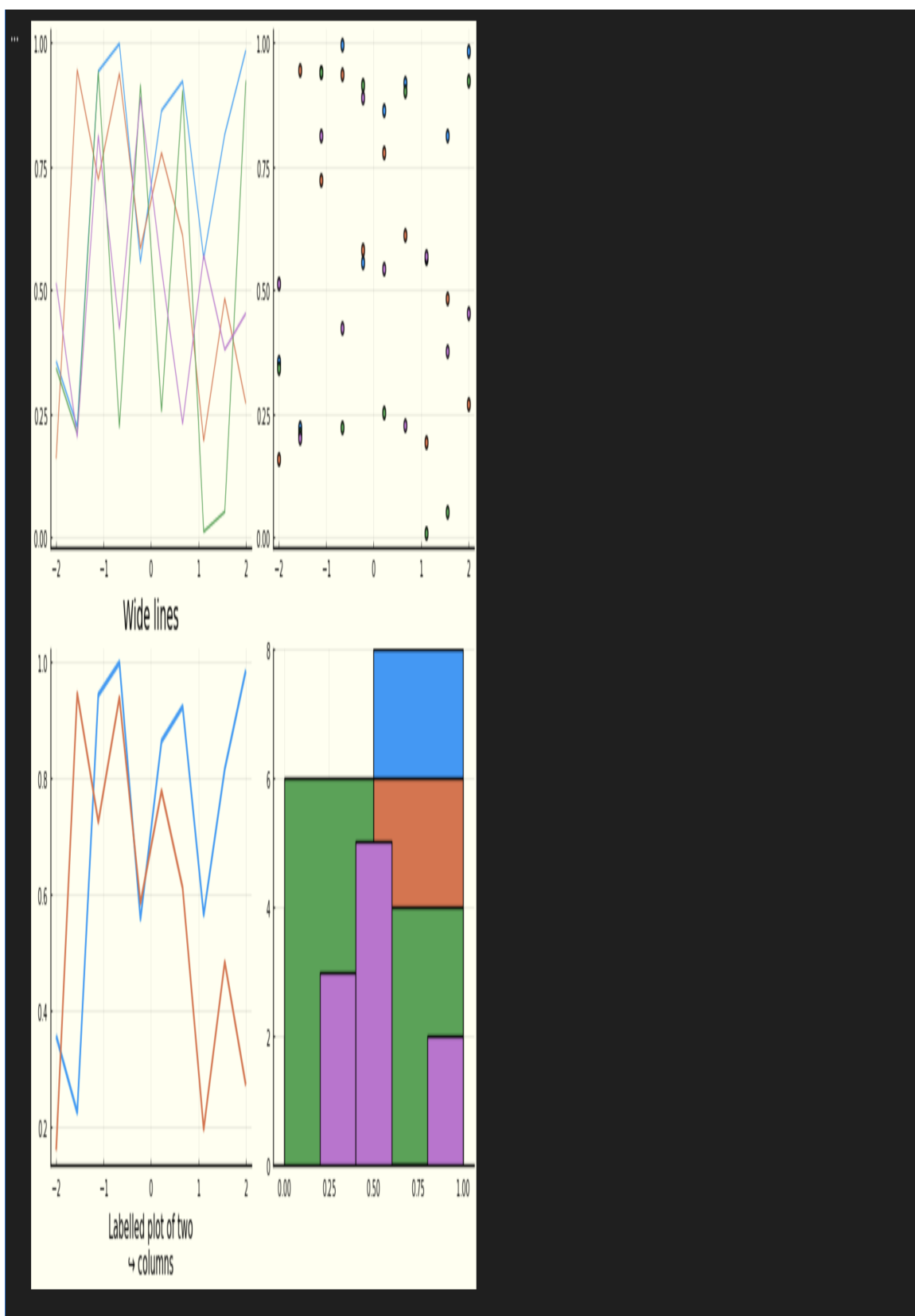


Рисунок 2.49: Объединение нескольких графиков в одной сетке

Обратите внимание, что атрибуты на отдельных графиках применяются к отдельным графикам, в то время как атрибуты в последнем вызове `plot` применяются ко всем графикам.

Разнообразные варианты представления данных (рис. [fig:050]):

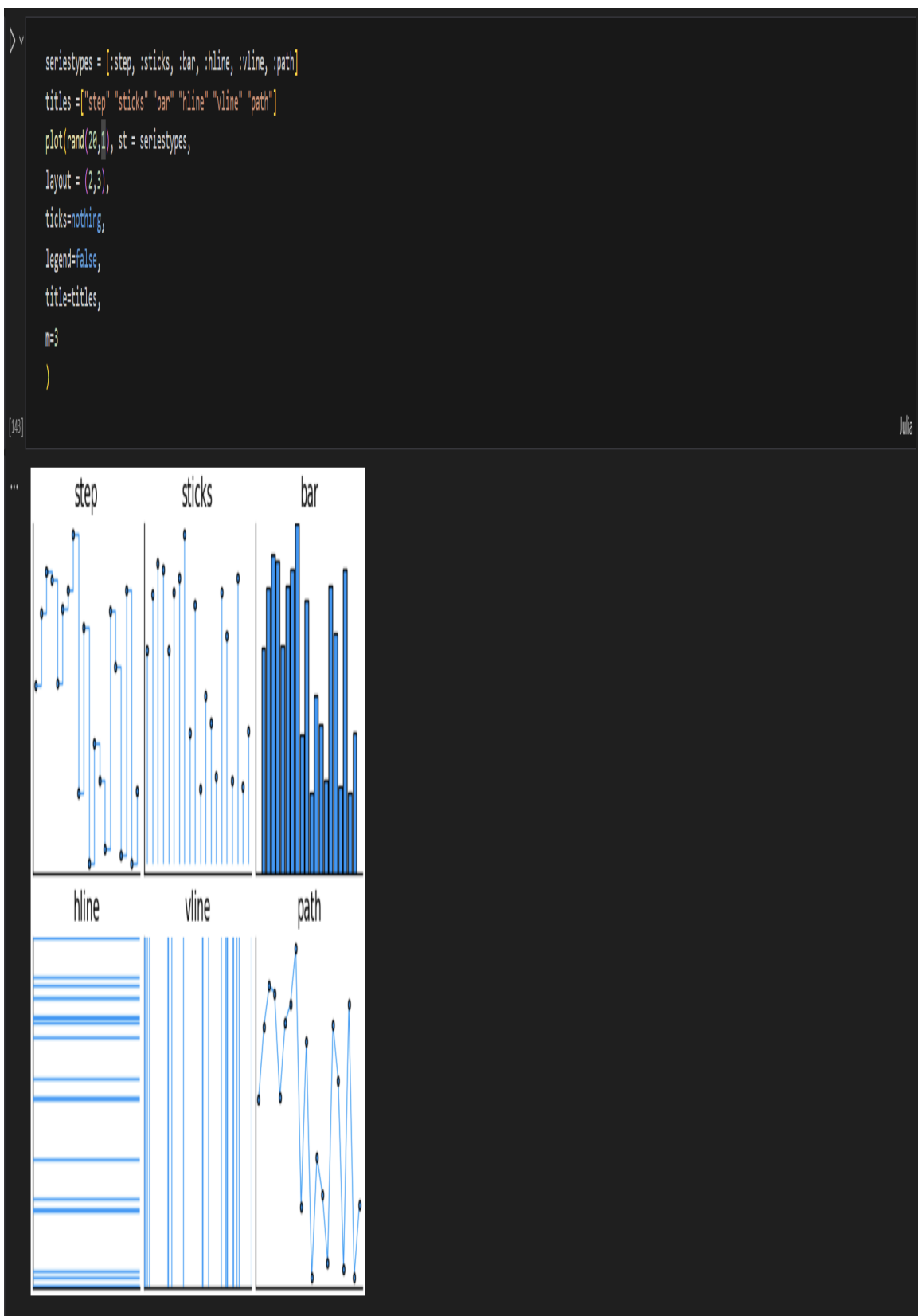


Рисунок 2.50: Разнообразные варианты представления данных

Применение макроса наиболее простой способ определения сложных макетов. Точные размеры могут быть заданы с помощью фигурных скобок, в противном случае пространство будет поровну разделено между графиками (рис. [fig:051]):

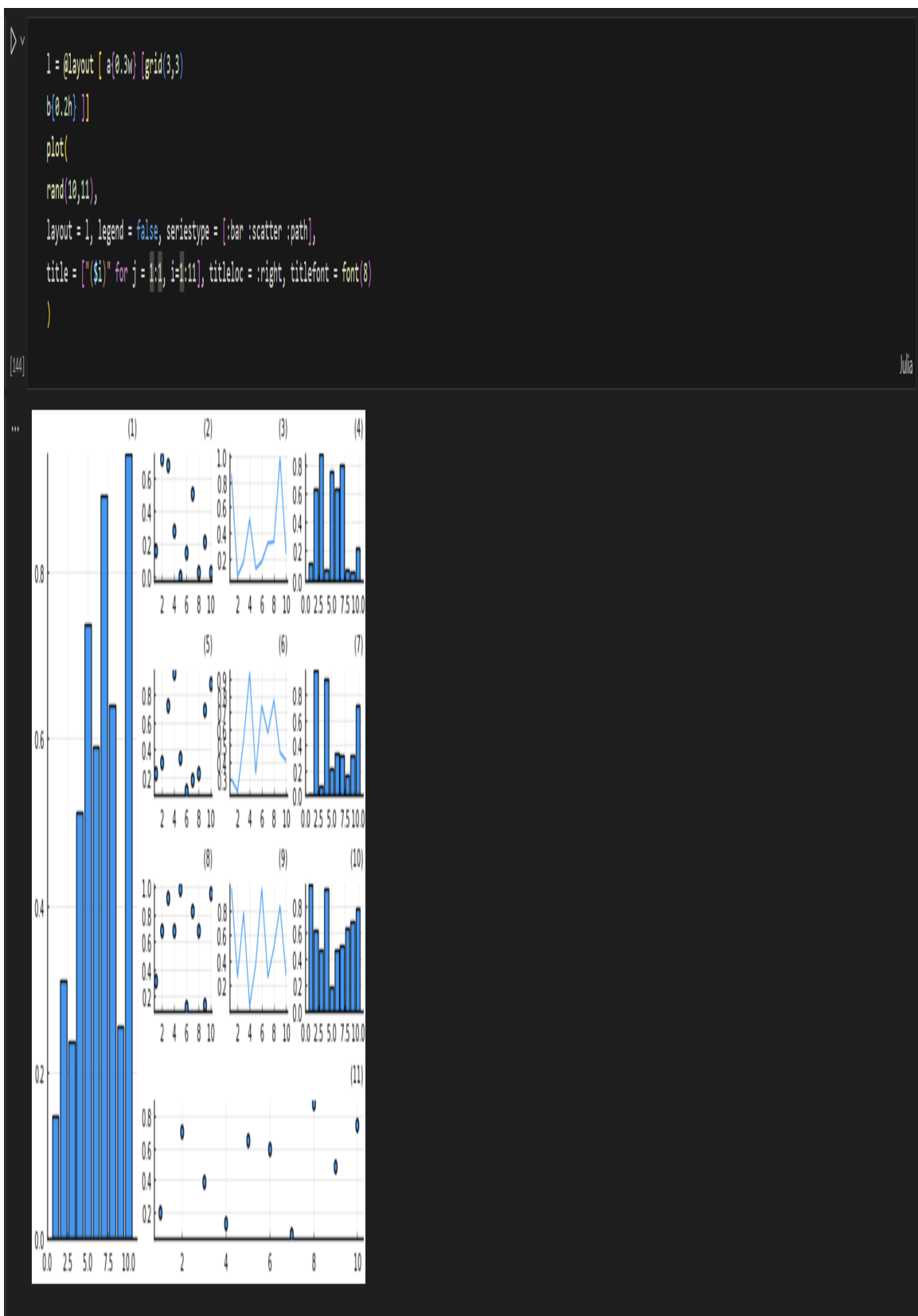


Рисунок 2.51: Демонстрация применения сложного макета для построения графиков

2.16 Самостоятельное выполнение

Выполнение задания №1 (рис. [fig:052]):

Задания

1. Постройте все возможные типы графиков (простые, точечные, гистограммы и т.д.) функции $y = \sin(x)$, $x = 0, 2\pi$. Отобразите все графики в одном графическом окне.

```

x = collect(range(0, 2*pi, length=50))
f(x) = sin(x)
types = [:line, :scatter, :step, :sticks, :bar, :hline, :vline, :path]
titles = ["line", "scatter", "step", "sticks", "bar", "hline", "vline", "path"]
plot(x, f, st=types, title=titles, layout=(2,4), size=(1800,600))

```

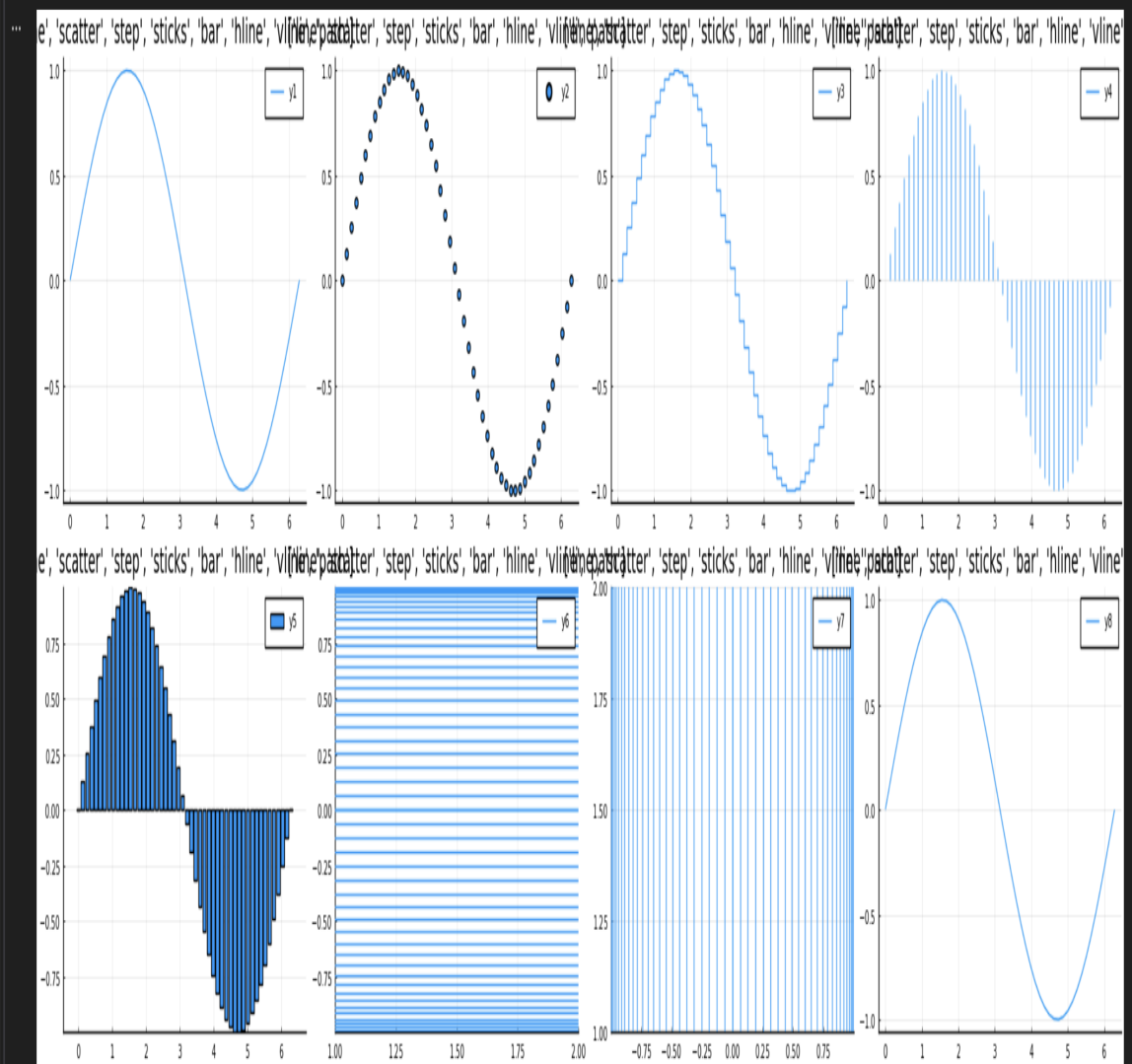


Рисунок 2.52: Решение задания №1

Выполнение задания №2 (рис. [fig:053]):

2. Постройте графики функции $y = \sin(x)$, $x = 0..2\pi$ со всеми возможными (сколько сможете вспомнить) типами оформления линий графика. Отобразите все графики в одном графическом окне.

```

titles = ["solid", "dash", "dot", "dashdot"]
p1 = Plots.plot(x, f, linestyle=solid)
p2 = Plots.plot(x, f, linestyle=dash)
p3 = Plots.plot(x, f, linestyle=dot)
p4 = Plots.plot(x, f, linestyle=dashdot)
Plots.plot(p1, p2, p3, p4, title=titles, layout=(2,2))

```

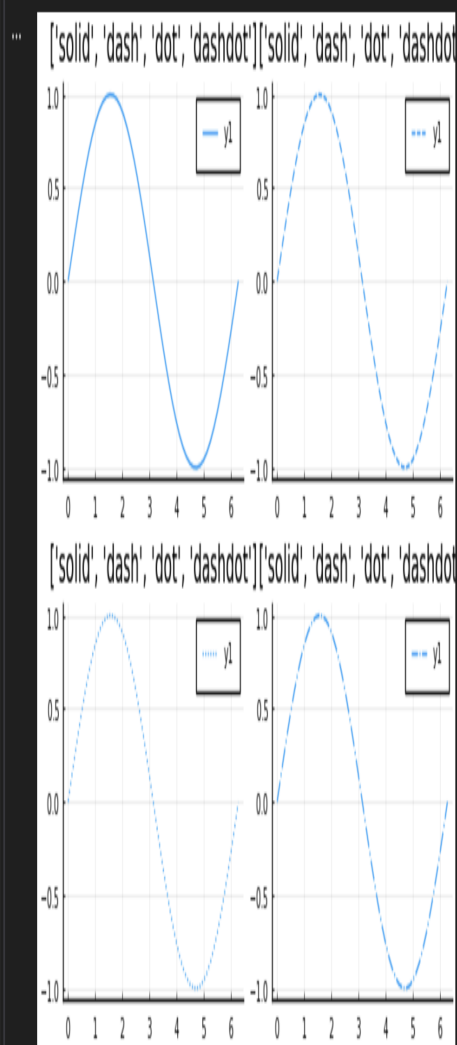


Рисунок 2.53: Решение задания №2

Выполнение задания №3 (рис. [fig:054]):

3. Постройте график функции $y(x) = \pi x^2 \ln(x)$, назовите оси соответственно. Пусть цвет рамки будет зелёным, а цвет самого графика — красным. Задайте расстояние между надписями и осями так, чтобы надписи полностью уместались в графическом окне. Задайте шрифт надписей. Задайте частоту отметок на осях координат.

```
f(x) = pi*x^2*log(x)
x = 0.1:0.01:6
plot(x, f(x),
      color = :red,          # цвет графика
      label = "y(x) =  $\pi x^2 \ln(x)$ ", # подпись в легенде
      xlabel = "x",          # подпись по оси x
      ylabel = "y(x) =  $\pi x^2 \ln(x)$ ", # подпись по оси y
      framestyle = :box,     # стиль рамки графика
      xticks = 0:0.5:6,     # частота отметок по оси x
      yticks = -50:50:200,  # частота отметок по оси y
      bordercolor = :green)  # цвет рамки графика
```

[148]

Julia

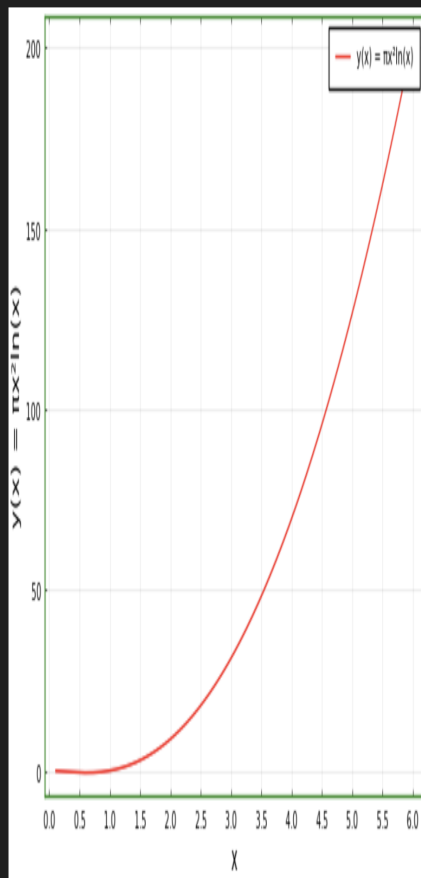


Рисунок 2.54: Решение задания №3

Выполнение задания №4 (рис. [fig:055]):

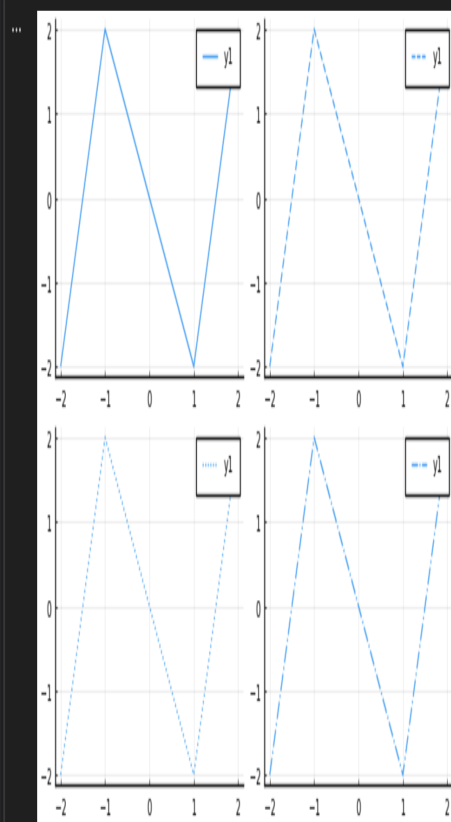
4. Задать вектор $x = (-2, -1, 0, 1, 2)$. В одном графическом окне (в 4-х подокнах) изобразите графически по точкам x значения функции $y(x) = x^3 - 3x$ в виде: – точек, – линий, – линий и точек, – кривой. Сохраните полученные изображения в файле `figure_familiya.png`, где вместо `familiya` укажите вашу фамилию.

```

x = [-2, -1, 0, 1, 2]
f(x) = x^3 - 3x
p1 = plot(x, f, lines=:solid)
p2 = plot(x, f, lines=:dash)
p3 = plot(x, f, lines=:dot)
p4 = plot(x, f, lines=:dashdot)
plot(p1, p2, p3, p4, layout=(2,2))

```

[149] Julia



```

savefig("figure_durdalyev.png")

```

[150] Julia

... "c:\\Users\\durda\\OneDrive\\Desktop\\Prog\\figure_durdalyev.png"

Рисунок 2.55: Решение задания №4

Выполнение задания №5 (рис. [fig:056] - рис. [fig:057]):

5. Задайте вектор $x = (3, 3.1, 3.2, \dots, 6)$. Постройте графики функций $y_1(x) = \pi x$ и $y_2(x) = \exp(x) \cos(x)$ в указанном диапазоне значений аргумента x следующим образом: – постройте оба графика разного цвета на одном рисунке, добавьте легенду и сетку для каждого графика; укажите недостатки у данного построения; – постройте аналогичный график с двумя осями ординат.

```

x = 3:0.1:6
y1(x) = pi*x
y2(x) = exp.(x) .* cos.(x)
plot(x, y1(x), label="y1(x) = πx", color=:blue, xlabel="x", ylabel="y", title="Графики y1(x) и y2(x)")
plot!(x, y2(x), label="y2(x) = exp(x) * cos(x)", color=:red)

```

[154]

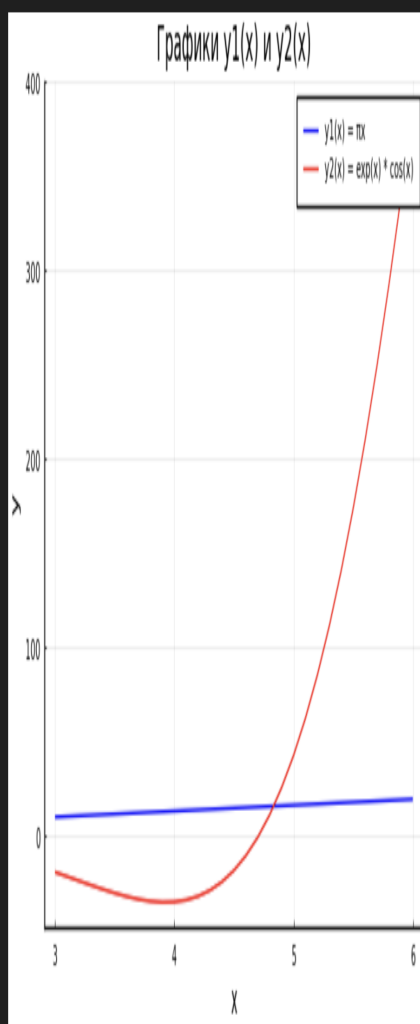


Рисунок 2.56: Решение задания №5

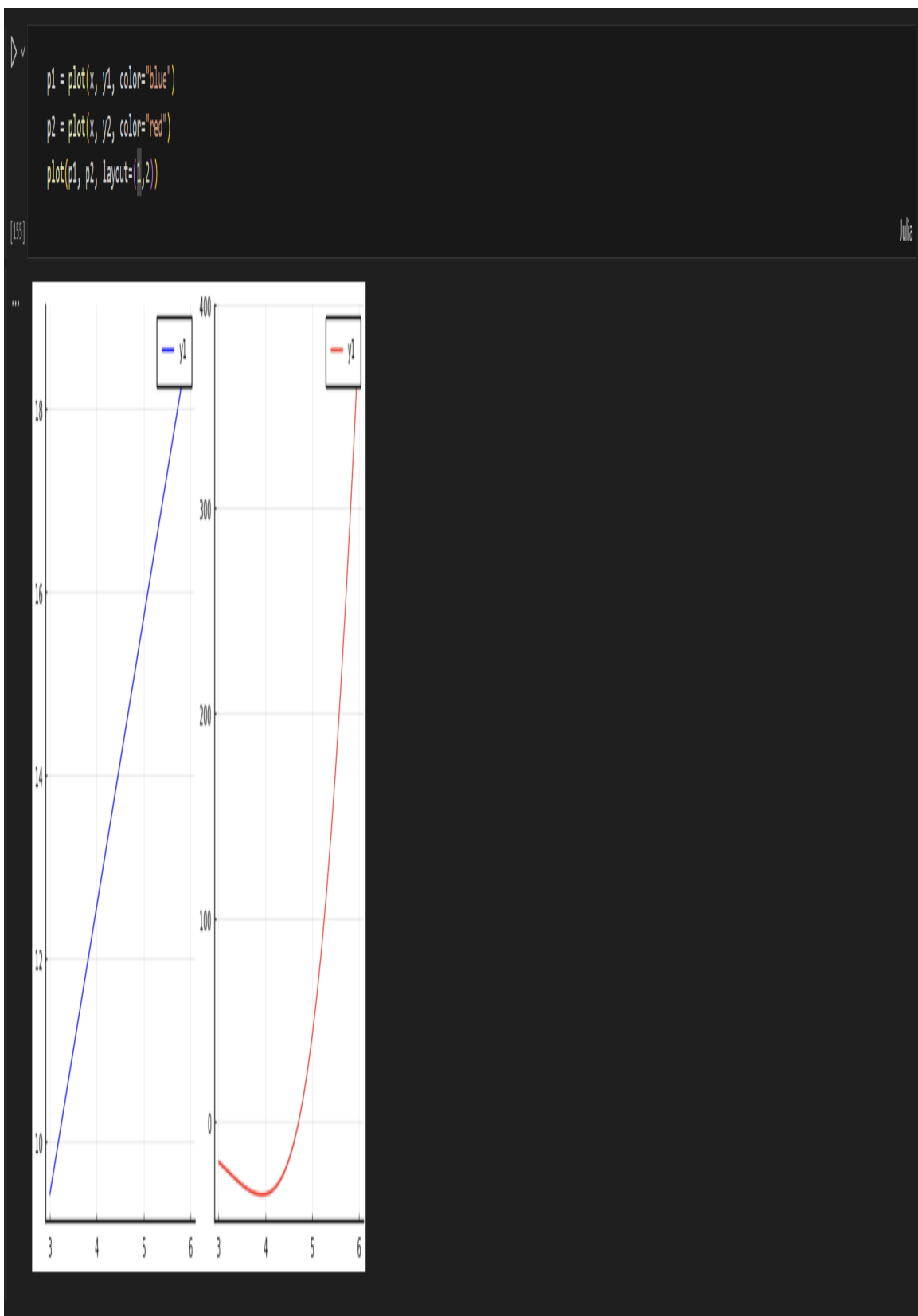


Рисунок 2.57: Решение задания №5

Выполнение задания №6 (рис. [fig:058]):

6. Постройте график некоторых экспериментальных данных (придумайте сами), учитывая ошибку измерения.

Д

```
time = 0:2:24  
osadki = [15, 21, 40, 32, 65, 12, 67, 34, 56, 45, 23, 18]  
osadki_error = [mean(sd*randn(n)) for sd in osadki]  
plot(time, osadki, label="Осадки",  
      xlabel="Время (часы)", ylabel="Осадки (мм)",  
      title="Экспериментальные данные с ошибкой измерения",  
      color=:green, yerr=osadki_error, marker=:o)
```

[156]

Julia

...

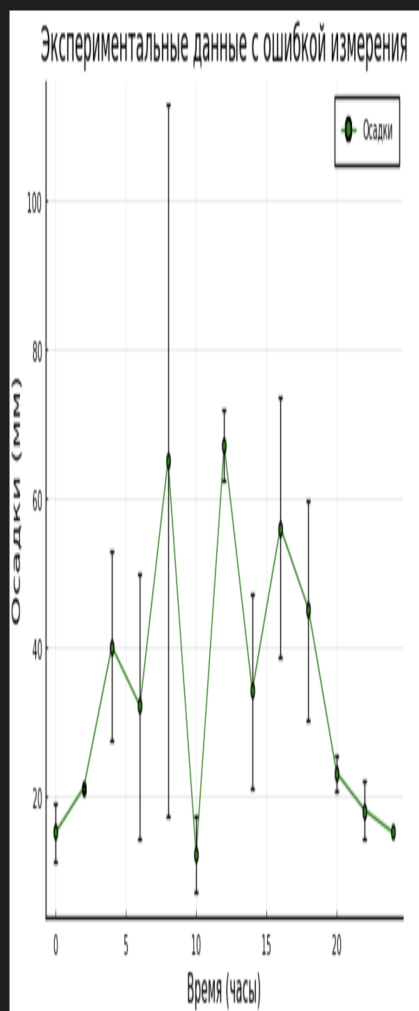


Рисунок 2.58: Решение задания №6

Выполнение задания №7 (рис. [fig:059]):

7. Постройте точечный график случайных данных. Подпишите оси, легенду, название графика

Д

```
x = rand(1:10, 20)
y = rand(1:10, 20)
scatter(x, y, label="Случайные данные", color=:purple, marker=:o,
        xlabel="Ось X", ylabel="Ось Y", title="Точечный график случайных данных")
```

[157]

Julia

...

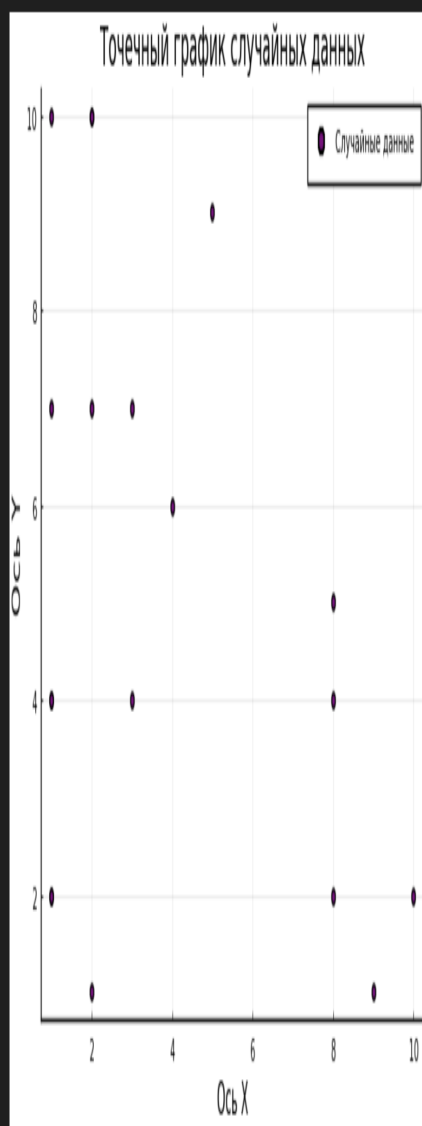


Рисунок 2.59: Решение задания №7

Выполнение задания №8 (рис. [fig:060]):

8. Постройте 3-мерный точечный график случайных данных. Подпишите оси, легенду, название графика.

```
x = rand(1:10, 20)
y = rand(1:10, 20)
z = rand(1:10, 20)

scatter(x, y, z, label="Случайные данные", color=:green, marker=:o,
        xlabel="x", ylabel="y", zlabel="z", title="3-мерный точечный график случайных данных")
```

[150]

Julia

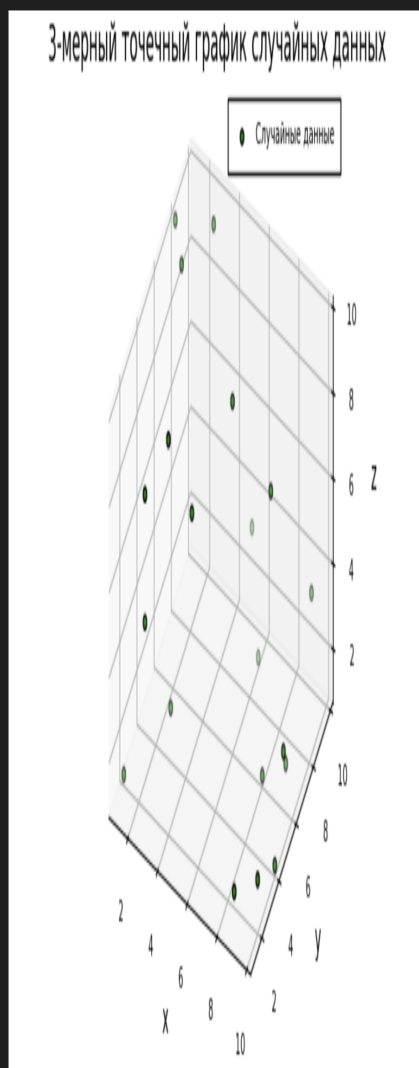


Рисунок 2.60: Решение задания №8

Выполнение задания №9 (рис. [fig:061]):

9. Создайте анимацию с построением синусоиды. То есть вы строите последовательность графиков синусоиды, постепенно увеличивая значение аргумента. После соедините их в анимацию.

```
x = 0:0.1:10
y_vals = sin.(x)
anim = @animate for i in 1:length(x)
    plot(x[1:i], y_vals[1:i], label="Синусоиды", color=:red, linewidth=2)
    xlabel("x")
    ylabel("sin(x)")
    title("Построение синусоиды")
end
gif(anim, "sin_wave_animation.gif", fps=10)
```

[159] Julia

Info: Saved animation to c:\Users\durda\OneDrive\Desktop\Proj\sin wave animation.gif
@ Plots C:\Users\durda\Julia\packages\Plots\8Zn83\src\animation.jl:156

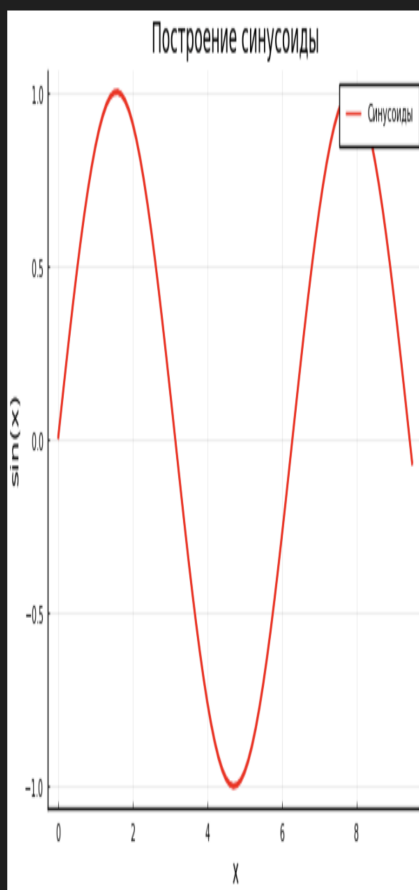


Рисунок 2.61: Решение задания №9

Выполнение задания №10 (рис. [fig:062] - рис. [fig:067]):

10. Постройте анимированную гипоциклоиду для 2 целых значений модуля k и 2 рациональных значений модуля k .

```

rr = 1
k = 10
n = 100

theta = collect(0:2*pi/100:2*pi+2*pi/100)
X = rr*k*cos.(theta)
Y = rr*k*sin.(theta)

anim = @animate for i in 1:n
    plt = plot(5, xlim=(-10,10), ylim=(-10,10), c="green", aspect_ratio=1, legend=false, framestyle="origin")
    plot!(plt, X, Y, c="blue", legend=false)
    t = theta[1:i]
    x = rr*(k-1)*cos.(t) + rr*cos.((k-1)*t)
    y = rr*(k-1)*sin.(t) - rr*sin.((k-1)*t)
    plot!(x, y, c="green")
    xc = rr*(k-1)*cos(t[end]) .+ rr*cos.(theta)
    yc = rr*(k-1)*sin(t[end]) .+ rr*sin.(theta)
    plot!(xc, yc, c="black")
    x1 = transpose([rr*(k-1)*cos(t[end]) x[end]])
    y1 = transpose([rr*(k-1)*sin(t[end]) y[end]])
    plot!(x1, y1, makershape="circle", markersize=4, c="black")
    scatter!([x[end]], [y[end]], c="green", markerstrokecolor="green")
end

gif(anim, "hypocycloid.gif")

```

[160]

Julia

Рисунок 2.62: Решение задания №10

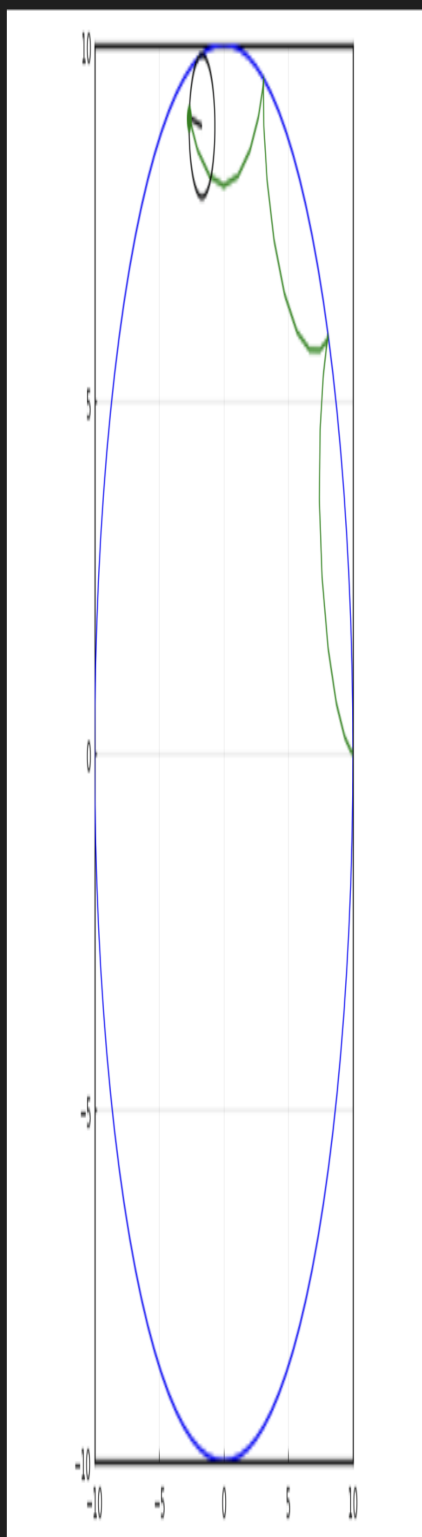


Рисунок 2.63: Решение задания №10

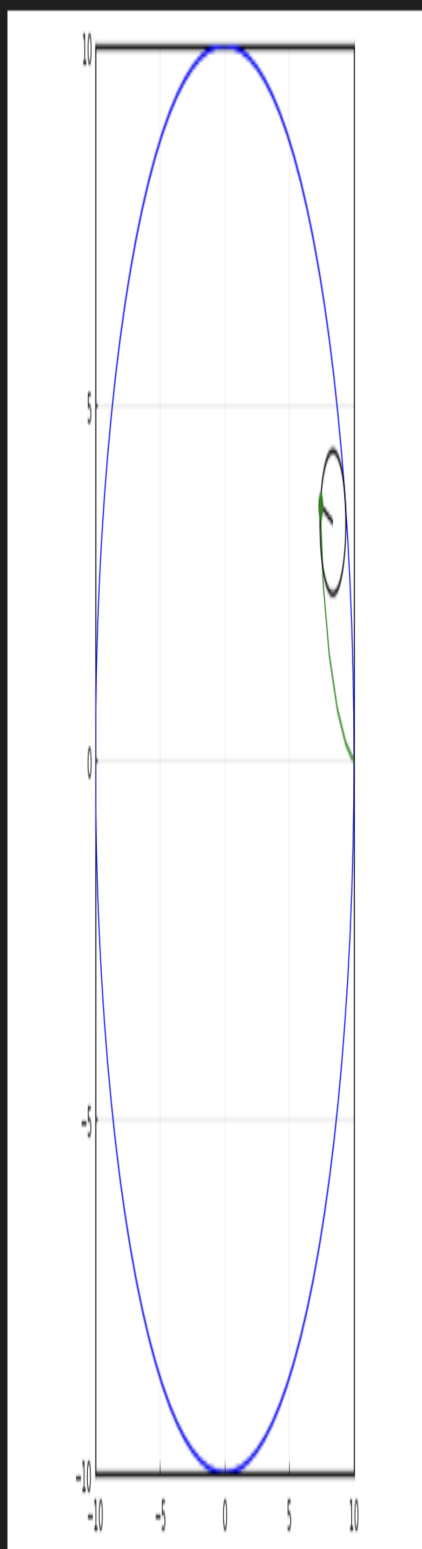


Рисунок 2.64: Решение задания №10

```

rr = 1
k = 4
n = 100
theta = collect(0:2*pi/100:2*pi+2*pi/100)
X = rr*k*cos.(theta)
Y = rr*k*sin.(theta)
anim = @animate for i in 1:n
    plt = plot(5, xlim=(-10,10), ylim=(-10,10), c="green", aspect_ratio=1, legend=false, framestyle=:origin)
    plot!(plt, X, Y, c="blue", legend=false)
    t = theta[1:i]
    x = rr*(k-1)*cos.(t) + rr*cos.((k-1)*t)
    y = rr*(k-1)*sin.(t) - rr*sin.((k-1)*t)
    plot!(x, y, c="green")
    xc = rr*(k-1)*cos(t[end]) + rr*cos.(theta)
    yc = rr*(k-1)*sin(t[end]) + rr*sin.(theta)
    plot!(xc, yc, c="black")
    x1 = transpose((rr*(k-1)*cos(t[end]) x[end]))
    y1 = transpose((rr*(k-1)*sin(t[end]) y[end]))
    plot!(x1, y1, markershape=:circle, markersize=4, c="black")
    scatter!([x[end]], [y[end]], c="green", markerstrokecolor="green")
end
gif(anim, "hypocycloid.gif")

```

[6]

✓ 30s

Julia

Рисунок 2.65: Решение задания №10

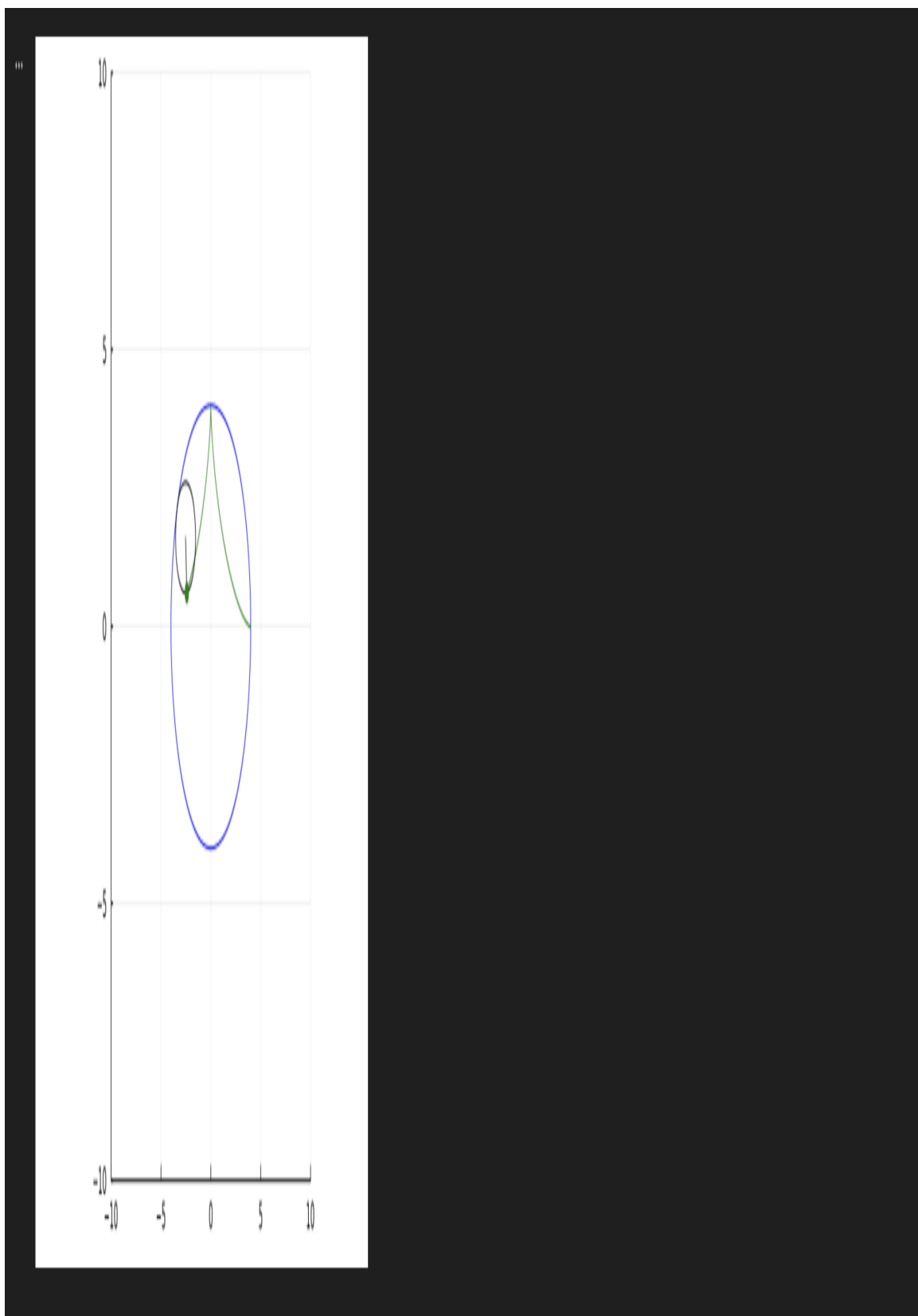


Рисунок 2.66: Решение задания №10

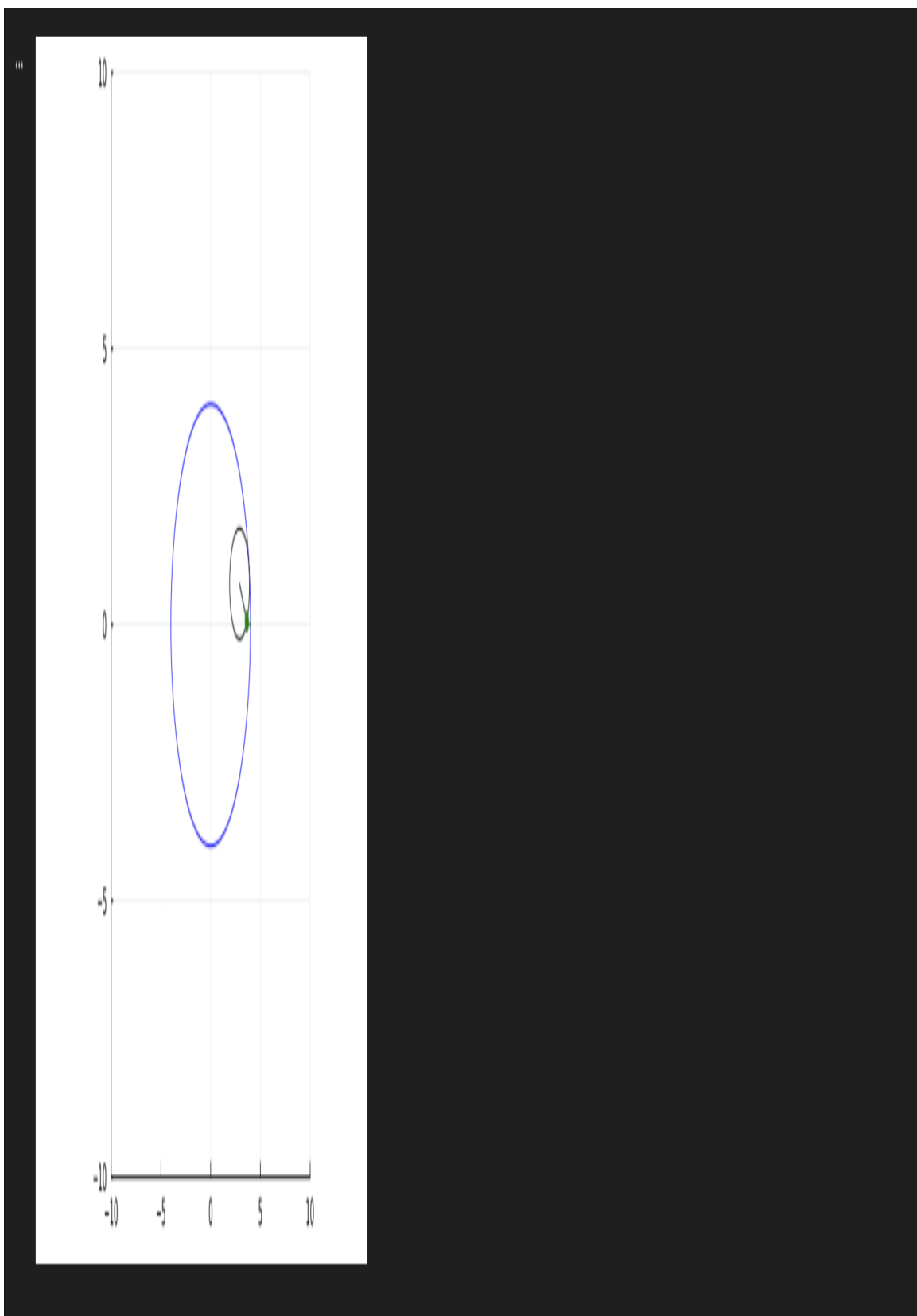


Рисунок 2.67: Решение задания №10

Выполнение задания №11 (рис. [fig:068] - рис. [fig:073]):

11. Постройте анимированную эпициклоиду для 2 целых значений модуля k и 2 рациональных значений модуля k .

```

rr = 1
k = 10
n = 100
theta = collect(0:2*pi/100:2*pi+2*pi/100)
X = rr*k*cos.(theta)
Y = rr*k*sin.(theta)
anim = @animate for i in 1:n
    plt = plot(5, xlim=(-10,10), ylim=(-10,10), c="green", aspect_ratio=1, legend=false, framestyle="origin")
    plot!(plt, X, Y, c="blue", legend=false)
    t = theta[k:i]
    x = rr*(k-1)*cos.(t) - rr*cos.((k-1)*t)
    y = rr*(k-1)*sin.(t) - rr*sin.((k-1)*t)
    plot!(x, y, c="green")
    xc = rr*(k-1)*cos(t[end]) .+ rr*cos.(theta)
    yc = rr*(k-1)*sin(t[end]) .+ rr*sin.(theta)
    plot!(xc, yc, c="black")
    x1 = transpose((rr*(k-1)*cos(t[end]) x[end]))
    y1 = transpose((rr*(k-1)*sin(t[end]) y[end]))
    plot!(x1, y1, markershape="circle", markersize=4, c="black")
    scatter!([x[end]], [y[end]], c="green", markerstrokecolor="green")
end
gif(anim, "hypocycloid.gif")

```

[162]

Julia

Рисунок 2.68: Решение задания №11

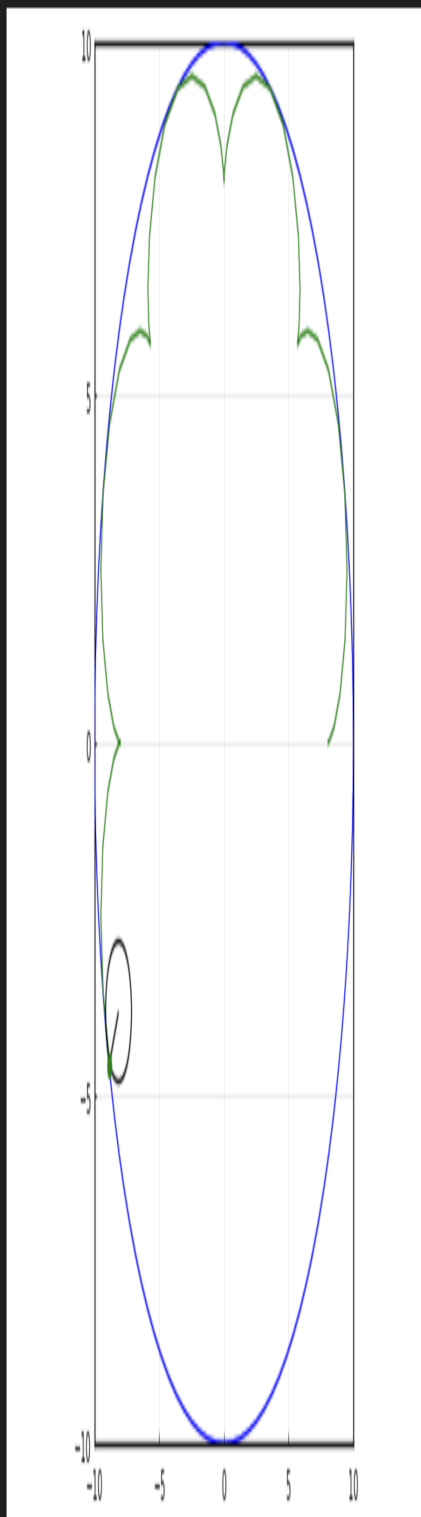


Рисунок 2.69: Решение задания №11

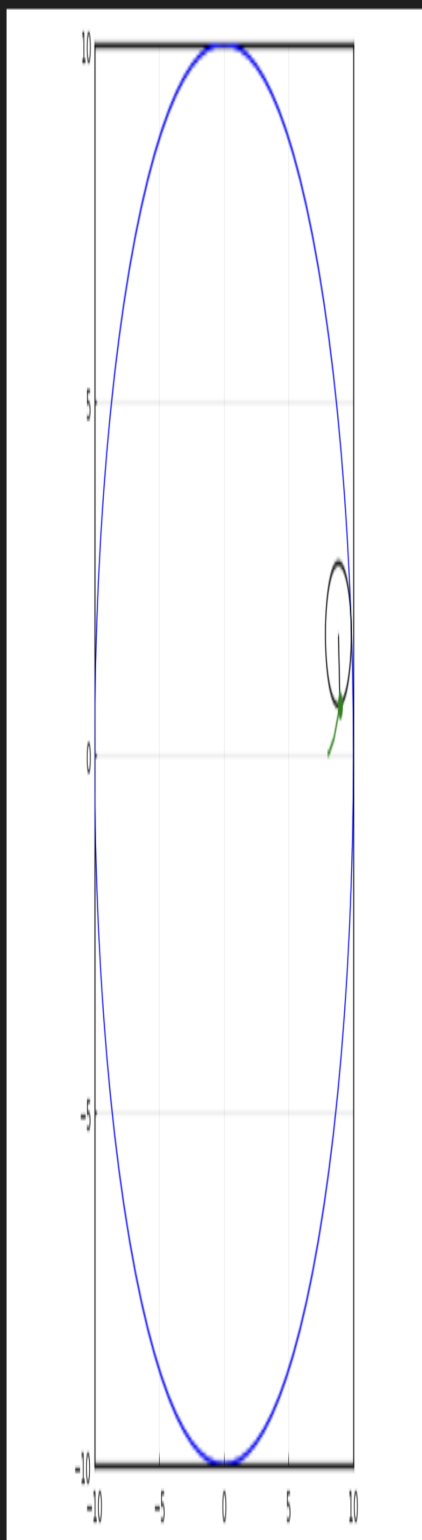


Рисунок 2.70: Решение задания №11

```

rr = 1
k = 19/3
n = 100
theta = collect(0:2*pi/100:2*pi+2*pi/100)
X = rr*k*cos.(theta)
Y = rr*k*sin.(theta)
anim = @animate for i in 1:n
    plt = plot(5, xlim=(-10,10), ylim=(-10,10), c="green", aspect_ratio=1, legend=false, framestyle="origin")
    plot!(plt, X, Y, c="blue", legend=false)
    t = theta[1:i]
    x = rr*(k-1)*cos.(t) - rr*cos.((k-1)*t)
    y = rr*(k-1)*sin.(t) - rr*sin.((k-1)*t)
    plot!(x, y, c="green")
    xc = rr*(k-1)*cos(t[end]) .+ rr*cos.(theta)
    yc = rr*(k-1)*sin(t[end]) .+ rr*sin.(theta)
    plot!(xc, yc, c="black")
    x1 = transpose((rr*(k-1)*cos(t[end]) x[end]))
    y1 = transpose((rr*(k-1)*sin(t[end]) y[end]))
    plot!(x1, y1, markershape="circle", markersize=4, c="black")
    scatter!([x[end]], [y[end]], c="green", markerstrokecolor="green")
end
gif(anim, "hypocycloid.gif")

```

[5] ✓ 6/s Julia

Рисунок 2.71: Решение задания №11

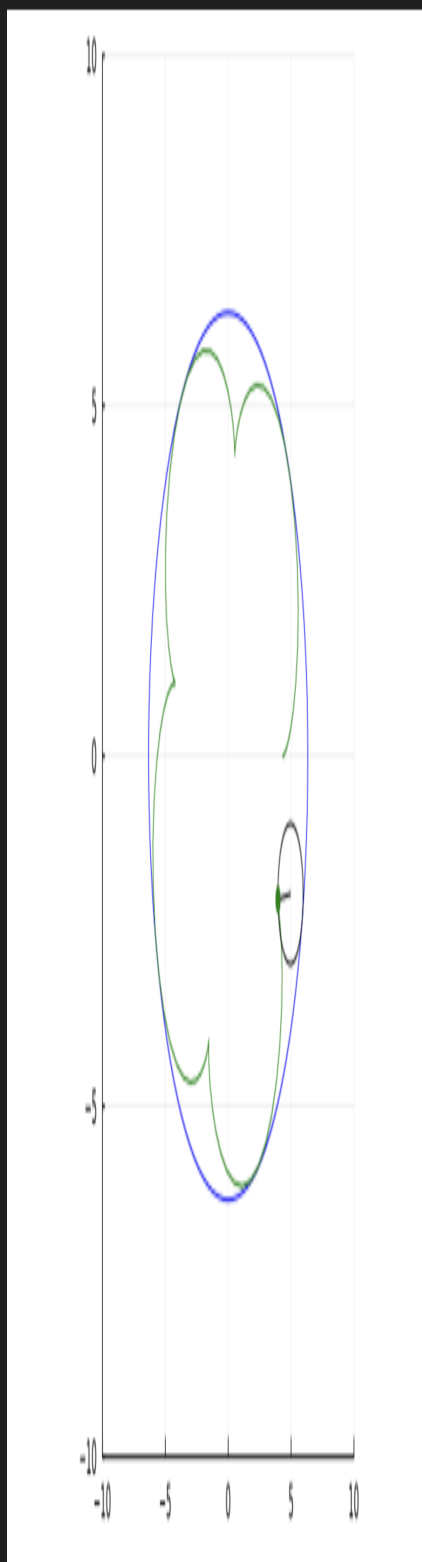


Рисунок 2.72: Решение задания №11

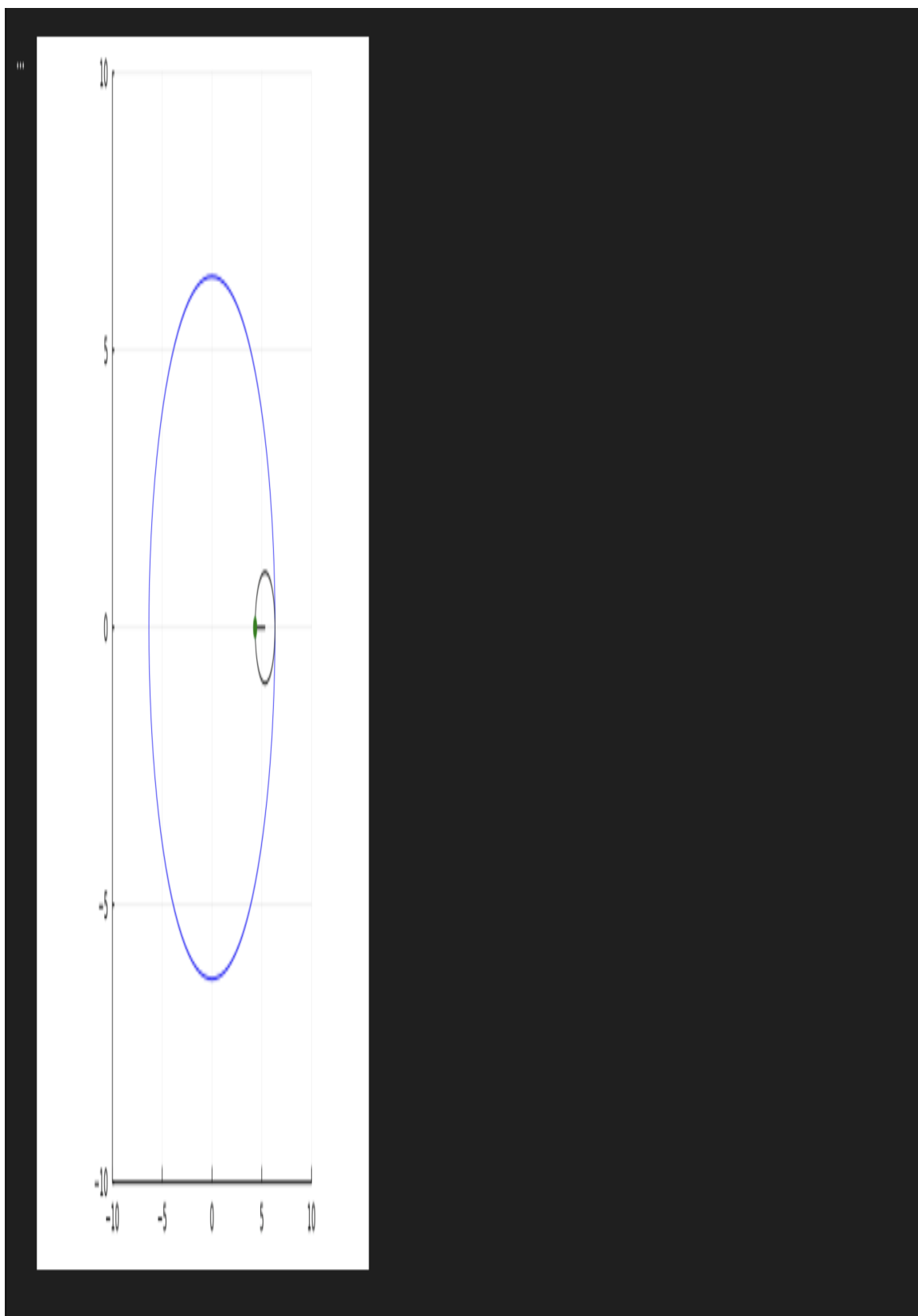


Рисунок 2.73: Решение задания №11

3 Выводы

В результате выполнения данной лабораторной работы я освоил синтаксис языка Julia для построения графиков.

Список литературы

- [1] *Julia 1.11 Documentation*. URL: <https://docs.julialang.org/en/v1/>.
- [2] *JuliaLang*. URL: <https://julialang.org/>.